

xv6 Labs 2021 实验报告（一）

Lab util: Xv6 and Unix Utilities

作者: Shalom0919

实验分支: util

本地提交: 1ef53b1

代码仓库: github.com/Shalom0919/xv6_labs

完成日期: 2026-07-29

官方评分: 100/100

1. 实验概述

本实验基于 MIT 6.S081 Fall 2021 的 [xv6-labs-2021](#) 仓库与 [util](#) 分支，目标是熟悉 xv6 的编译、启动、用户程序集成方式，以及进程、管道和文件系统相关系统调用。按照官方要求完成以下五个用户态工具：

- [sleep](#)：按指定 tick 数休眠。
- [pingpong](#)：父子进程通过两个管道交换一个字节。
- [primes](#)：用进程和管道实现并发素数筛。
- [find](#)：递归查找目录树中的同名文件。
- [xargs](#)：逐行读取标准输入，组装参数并执行命令。

官方实验说明：[MIT 6.S081 – Lab: Xv6 and Unix utilities](#)

1.1 实验环境

项目	配置
宿主环境	Windows + WSL2 Ubuntu, x86_64
编译器	riscv64-linux-gnu-gcc 15.2.0
模拟器	QEMU system-riscv64 10.2.1
Python	Python 3.14.4, 用于官方评分器
官方基线	origin/util , 提交 f654383
实验成果	util , 提交 1ef53b1

1.2 Makefile 集成

五个程序加入 `UPROGS`，从而在构建 `fs.img` 时被复制到 xv6 文件系统：

```
[makefile]
UPROGS=\
...
$U/_sleep\
$U/_pingpong\
$U/_primes\
$U/_find\
$U/_xargs\
```

2. sleep: 定时休眠

2.1 设计

程序检查命令行参数数量，用 `atoi` 将字符串转换为整数，再调用 xv6 已有的 `sleep` 系统调用。参数缺失时打印用法并以非零状态退出；正常路径最后显式调用 `exit(0)`。

2.2 关键代码

```
[c]
int
main(int argc, char *argv[])
{
    if(argc != 2){
        fprintf(2, "usage: sleep ticks\n");
        exit(1);
    }

    sleep(atoi(argv[1]));
    exit(0);
}
```

2.3 结果分析

官方测试分别验证无参数处理、休眠后正常返回，以及是否确实进入内核 `sys_sleep`。三个测试均通过。手工执行 `sleep 2` 后，shell 在相应 tick 数后继续执行 `echo sleep-ok`。

3. pingpong: 双向进程通信

3.1 设计

父子进程间建立两个方向相反的管道。父进程先向子进程发送 1 字节，子进程阻塞读取后输出 `received ping` 并回写；父进程收到回包后输出 `received pong`。两端及时关闭无用文件描述符，父进程最后用 `wait` 回收子进程。

3.2 关键代码

```
[c]
int parent_to_child[2];
int child_to_parent[2];
char byte = 'x';
pipe(parent_to_child);
pipe(child_to_parent);

int pid = fork();
if(pid == 0){
    close(parent_to_child[1]);
    close(child_to_parent[0]);
    read(parent_to_child[0], &byte, 1);
    printf("%d: received ping\n", getpid());
    write(child_to_parent[1], &byte, 1);
    exit(0);
}

close(parent_to_child[0]);
close(child_to_parent[1]);
write(parent_to_child[1], &byte, 1);
read(child_to_parent[0], &byte, 1);
printf("%d: received pong\n", getpid());
wait(0);
```

3.3 结果分析

实际输出中子进程先打印 `ping`，父进程后打印 `pong`。顺序由管道阻塞语义保证，不依赖调度时序。官方 `pingpong` 测试通过。

4. primes：并发素数筛

4.1 设计

主进程把整数 2 至 35 写入第一个管道。每一级筛选进程将收到的第一个整数视为素数并打印，再创建下一级进程，把不能被该素数整除的整数写入右侧管道。左侧写端全部关闭后，`read` 返回 0，EOF 逐级传播；每级父进程用 `wait` 保证整个管线退出后再结束。

4.2 关键代码

```
[c]
static void
sieve(int input)
{
    int prime;
    if(read(input, &prime, sizeof(prime)) != sizeof(prime)){
        close(input);
        exit(0);
    }
    printf("prime %d\n", prime);

    int output[2];
    pipe(output);
    int pid = fork();
    if(pid == 0){
        close(input);
        close(output[1]);
        sieve(output[0]);
    }

    close(output[0]);
    int value;
    while(read(input, &value, sizeof(value)) == sizeof(value))
        if(value % prime != 0)
            write(output[1], &value, sizeof(value));
    close(output[1]);
    wait(0);
    exit(0);
}
```

4.3 结果分析

运行结果依次得到 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 与 2 至 35 范围内的理论素数集合一致。关闭不用的管道端点是避免资源耗尽和保证 EOF 到达的关键。

5. find: 递归目录遍历

5.1 设计

程序使用 `open` 和 `fstat` 判断路径类型。普通文件比较 `basename`；目录则逐项读取 `struct dirent`，拼接子路径并递归。跳过空目录项以及 `..`，避免递归环；同时检查 512 字节路径缓冲区边界。

5.2 关键代码

```
[c]
while(read(fd, &de, sizeof(de)) == sizeof(de)){
    if(de.inum == 0)
        continue;
    memmove(name, de.name, DIRSIZ);
    name[DIRSIZ] = 0;
    if(strcmp(name, ".") == 0 || strcmp(name, "..") == 0)
        continue;
    find(child, target);
}
```

5.3 结果分析

官方测试覆盖当前目录和多层递归目录，两项均通过。目录项名称最多为 `DIRSIZ` 字节，显式补零后再使用 `strcmp`，避免把非终止字符数组误当作普通 C 字符串。

6. xargs: 标准输入到参数向量

6.1 设计

程序每次从标准输入读取一个字符，遇到换行符后把该行按空格或制表符切分，将得到的参数追加到原始命令 `argv`。随后 `fork`：子进程 `exec`，父进程 `wait` 后继续处理下一行。参数数组始终预留最后一个空指针，并受 `MAXARG` 限制。

6.2 关键代码

```
[c]
while(read(0, &ch, 1) == 1){
    if(ch == '\n'){
        run_line(line, length, argc - 1, argv + 1);
        length = 0;
    } else if(length < sizeof(line) - 1){
        line[length++] = ch;
    } else {
        fprintf(2, "xargs: input line too long\n");
        exit(1);
    }
}

[c]
int pid = fork();
if(pid == 0){
    exec(argv[0], argv);
    fprintf(2, "xargs: exec %s failed\n", argv[0]);
    exit(1);
}
wait(0);
```

6.3 结果分析

`echo hello xv6 | xargs echo prefix` 输出 `prefix hello xv6`，说明固定参数与输入参数顺序正确。官方 `xargstest.sh` 通过 `find | xargs grep` 找到三处 `hello`，验证了逐行执行、参数切分和父子进程同步。

7. 运行结果截图

7.1 xv6 内手工运行

下面截图来自本次实际执行 `make qemu` 的原始日志，包含内核启动和五个工具的运行结果。

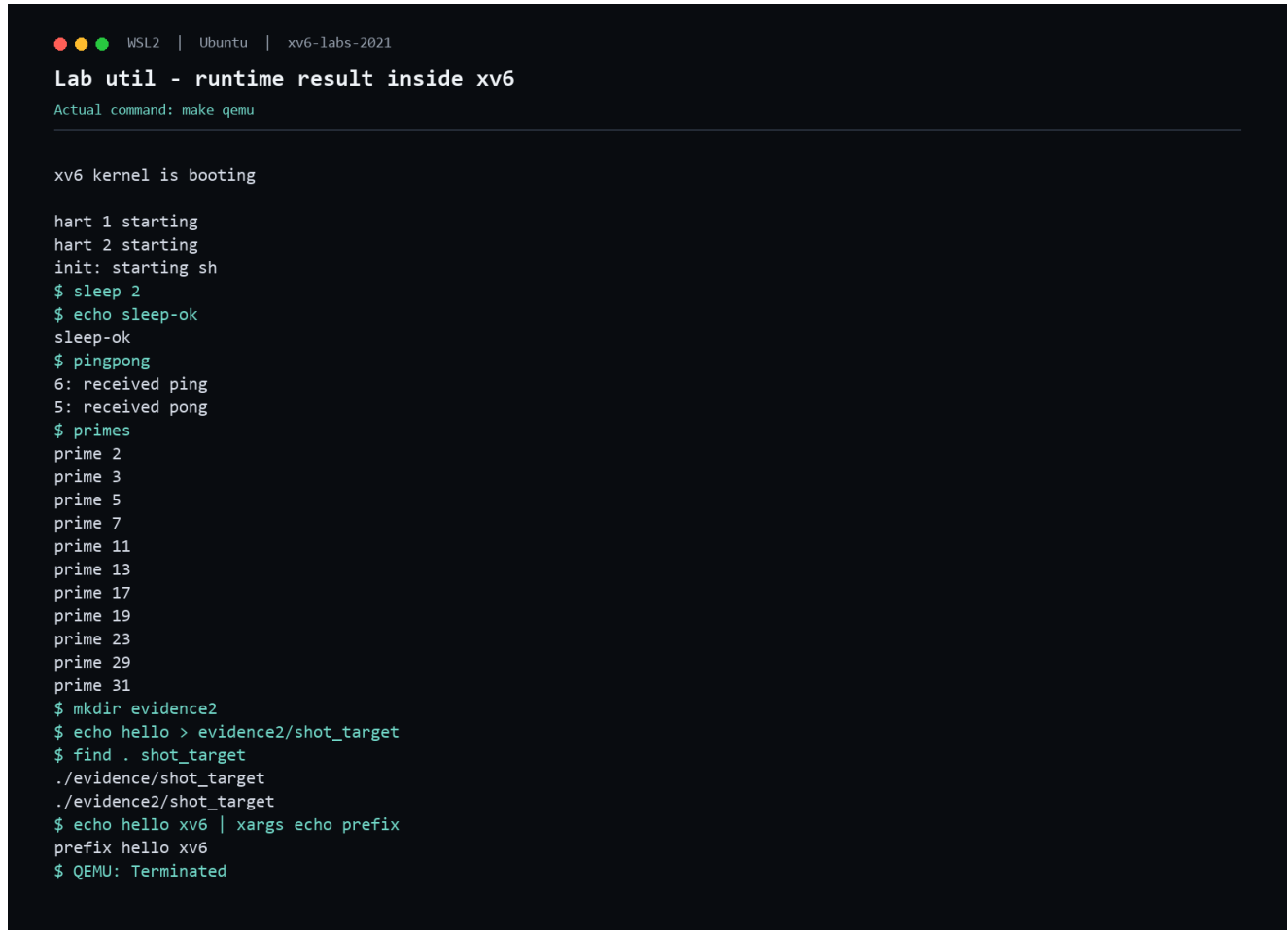
A terminal window with a dark background and light green text. The window title is 'WSL2 | Ubuntu | xv6-labs-2021'. The main title is 'Lab util - runtime result inside xv6'. Below it, it says 'Actual command: make qemu'. The output shows the xv6 kernel booting, followed by two hart units starting. The init process starts a shell. Then, several tools are run: 'sleep 2', 'echo sleep-ok', 'pingpong' (showing ping and pong received), 'primes' (listing prime numbers up to 31), 'mkdir evidence2', 'echo hello > evidence2/shot_target', 'find . shot_target' (showing the file path), and 'echo hello xv6 | xargs echo prefix' (showing the prefix). The final line is '\$ QEMU: Terminated'.

图 1: xv6 内依次验证 `sleep`、`pingpong`、`primes`、`find` 和 `xargs`。

7.2 官方评分

下面截图来自本次实际执行 `make grade` 的原始日志末段。所有测试均为 `OK`，最终得分 `100/100`。


```
WSL2 | Ubuntu | xv6-labs-2021

Lab util - official grading result

Actual command: make grade

$ make qemu-gdb
sleep, no arguments: OK (7.3s)
== Test sleep, returns ==
$ make qemu-gdb
sleep, returns: OK (0.6s)
== Test sleep, makes syscall ==
$ make qemu-gdb
sleep, makes syscall: OK (1.0s)
== Test pingpong ==
$ make qemu-gdb
pingpong: OK (0.9s)
== Test primes ==
$ make qemu-gdb
primes: OK (1.1s)
== Test find, in current directory ==
$ make qemu-gdb
find, in current directory: OK (1.0s)
== Test find, recursive ==
$ make qemu-gdb
find, recursive: OK (1.2s)
== Test xargs ==
$ make qemu-gdb
xargs: OK (1.0s)
== Test time ==
time: OK
Score: 100/100
```

图 2：MIT 官方评分器测试结果。

测试项	分值	结果
sleep, no arguments	5	OK
sleep, returns	5	OK
sleep, makes syscall	10	OK
pingpong	20	OK
primes	20	OK
find, current directory	10	OK
find, recursive	10	OK
xargs	19	OK
time	1	OK
总分	100	100/100

8. 2026 工具链兼容问题

首次构建后 QEMU 串口没有启动输出。指令跟踪表明入口地址 `0x80000010` 被 GCC 15 默认架构汇编为 `c.mul`。当前编译器默认启用了 Zcb、V、B 等新 RISC-V 扩展，而 xv6 虚拟 CPU 没有启用这些扩展，因此在启动早期触发非法指令。

解决办法是在 Makefile 中显式限定目标 ISA，并处理现代 GCC 新增但不影响 xv6 逻辑的告警：

```
[makefile]
CFLAGS = -Wall -Werror -Wno-infinite-recursion \
         -Wno-incompatible-pointer-types -O -fno-omit-frame-pointer -ggdb
CFLAGS += -march=rv64gc
ASFLAGS = -march=rv64gc
```

Python 3.14 已移除 `pipes.quote`，评分器中将其替换为等价的 `shlex.quote`。以上调整仅用于保证 2021 教学代码在 2026 工具链上可复现，不修改评分条件和实验逻辑。

9. 如何自行验证官方评分

在 PowerShell 中执行：

```
[powershell]
wsl -d Ubuntu
cd /mnt/c/Users/Aemeath/Helper/2026/OS_Design/xv6-labs-2021
git checkout util
make clean
make grade
```

通过标准有两个：

- 1 命令退出码为 0；
- 2 输出最后一行是 **Score: 100/100**，且各项均显示 OK。

若只验证某一项，可执行：

```
[bash]
./grade-lab-util sleep
./grade-lab-util pingpong
./grade-lab-util primes
./grade-lab-util find
./grade-lab-util xargs
```

10. 结论

本实验完成了 xv6 用户态程序的完整开发闭环：Makefile 集成、RISC-V 交叉编译、QEMU 启动、系统调用组合、官方测试和手工运行验证。五个必做任务全部完成，官方评分为 100/100。实现过程中最关键的正确性条件是关闭不用的文件描述符、理解管道阻塞与 EOF、回收子进程、跳过 `.` 和 `..`，以及严格控制路径和参数数组边界。

可选挑战（uptime、正则 find、扩展 shell）不属于本次必做验收范围，未计入本报告。

xv6 Labs 2021 实验报告（二）

Lab syscall: System Calls

作者: Shalom0919

实验分支: syscall

本地提交: b312e1d

代码仓库: github.com/Shalom0919/xv6_labs

完成日期: 2026-07-29

官方评分: 35/35

1. 实验概述

本实验基于 MIT 6.S081 Fall 2021 的 `xv6-labs-2021` 仓库 `syscall` 分支，目标是理解用户程序通过 RISC-V `ecall` 进入内核、系统调用分派、参数读取和结果返回的完整路径，并新增以下两个系统调用：

- `trace(mask)`：按位掩码跟踪当前进程及其后代的系统调用。
- `sysinfo(info)`：向用户空间返回空闲物理内存字节数和当前进程数。

官方实验说明：[MIT 6.S081 - Lab: System calls](#)

1.1 实验环境

项目	配置
宿主环境	Windows + WSL2 Ubuntu, x86_64
目标架构	RISC-V 64
编译工具链	<code>riscv64-linux-gnu-gcc</code>
模拟器	<code>qemu-system-riscv64</code>
官方基线	<code>origin/syscall</code>
实验成果	<code>syscall</code> 分支，提交 <code>b312e1d</code>

1.2 系统调用路径

用户程序调用 `trace()` 或 `sysinfo()` 后，`user/usys.pl` 生成的汇编桩把系统调用编号写入寄存器 `a7`，再执行 `ecall`。内核陷入处理代码最终调用 `syscall()`，通过编号索引分派表，执行 `sys_trace()` 或 `sys_sysinfo()`。返回值写入陷阱帧的 `a0`，用户程序恢复执行后即可获得结果。

2. 用户接口与系统调用注册

2.1 用户态声明和汇编桩

在 `user/user.h` 中预声明 `struct sysinfo` 并增加函数原型，避免用户程序依赖内核实现细节：

```
[c]
struct sysinfo;

int trace(int);
int sysinfo(struct sysinfo *);
```

在 `user/usys.pl` 中增加入口。构建阶段会据此生成执行 `ecall` 的 `user/usys.S`：

```
[perl]
entry("trace");
entry("sysinfo");
```

2.2 编号与内核分派

在 `kernel/syscall.h` 中为两个调用分配唯一编号：

```
[c]
#define SYS_trace 22
#define SYS_sysinfo 23
```

在 `kernel/syscall.c` 中声明内核处理函数并加入分派表：

```
[c]
extern uint64 sys_trace(void);
extern uint64 sys_sysinfo(void);

static uint64 (*syscalls[])(void) = {
    /* existing entries */
    [SYS_trace] sys_trace,
    [SYS_sysinfo] sys_sysinfo,
};
```

同时在 `Makefile` 的 `UPROGS` 中加入 `_trace` 和 `_sysinfotest`，使它们被写入 `xv6` 文件系统镜像。

2.3 分析

系统调用必须同时完成用户声明、汇编桩、调用编号和内核分派四层注册。缺少任意一层都会分别表现为编译失败、链接失败、未知系统调用或无法进入实现函数。本实验通过这条链路验证了 `xv6` 将用户 API 与内核服务解耦的方式。

3. trace 系统调用

3.1 保存跟踪掩码

在 `struct proc` 中增加进程私有的 `trace_mask`。创建进程槽时初始化为 0，释放进程时清零，防止进程槽复用后继承旧状态：

```
[c]
struct proc {
    /* existing fields */
    int trace_mask;
};
```

`sys_trace()` 使用 `argint()` 读取第 0 个参数，并保存到当前进程：

```
[c]
uint64
sys_trace(void)
{
    int mask;

    if(argint(0, &mask) < 0)
        return -1;
    myproc()->trace_mask = mask;
    return 0;
}
```

3.2 fork 继承

实验要求调用者随后创建的所有子进程继续使用相同掩码，因此在 `fork()` 中复制该字段：

```
[c]
safestrcpy(np->name, p->name, sizeof(p->name));
np->trace_mask = p->trace_mask;
```

掩码属于进程状态而不是全局状态，所以未调用 `trace()` 的其他进程不会受到影响。

3.3 返回前输出

`syscall()` 完成分派后检查对应位。如果第 `num` 位为 1，就输出 PID、调用名称和返回值：

```
[c]
p->trapframe->a0 = syscalls[num]();
if(p->trace_mask & (1 << num))
    printf("%d: syscall %s -> %d\n",
        p->pid, syscall_names[num],
        (int)p->trapframe->a0);
```

输出发生在系统调用完成之后，因此 `a0`

已包含真实返回值。名称数组使用系统调用编号作为索引，避免复杂的条件分支。

3.4 结果分析

执行 `trace 32 grep hello README` 时，32 等于 `1 << SYS_read`，运行结果仅包含 `read`。官方测试还验证了跟踪全部调用、不跟踪任何调用以及 `fork()` 后代继承掩码，四项均为 OK。

4. sysinfo 系统调用

4.1 统计空闲物理内存

物理内存分配器以单向链表维护空闲页。`freemem()` 在持有 `kmem.lock` 时遍历链表，每个节点代表一个 `PGSIZE` 字节的页面：

```
[c]
uint64
freemem(void)
{
    uint64 bytes = 0;
    struct run *r;

    acquire(&kmem.lock);
    for(r = kmem.freelist; r; r = r->next)
        bytes += PGSIZE;
    release(&kmem.lock);
    return bytes;
}
```

锁保证统计期间链表不会被其他 CPU 上的 `kalloc()` 或 `kfree()` 修改。

4.2 统计活动进程

`nproc()` 遍历固定长度的进程表，并分别获取每个进程的锁。状态不为 `UNUSED` 的槽均计入结果，包括 `USED`、`SLEEPING`、`RUNNABLE`、`RUNNING` 和 `ZOMBIE`：

```
[c]
uint64
nproc(void)
{
    uint64 n = 0;
    struct proc *p;

    for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        if(p->state != UNUSED)
            n++;
        release(&p->lock);
    }
    return n;
}
```

4.3 安全返回用户空间

`sys_sysinfo()` 读取用户指针，在内核栈上构造结果，再通过 `copyout()` 写入当前进程页表：

```
[c]
uint64
sys_sysinfo(void)
{
    uint64 addr;
    struct sysinfo info;
    struct proc *p = myproc();

    if(argaddr(0, &addr) < 0)
        return -1;
    info.freemem = freemem();
    info.nproc = nproc();
    if(copyout(p->pagetable, addr,
              (char *)&info, sizeof(info)) < 0)
        return -1;
    return 0;
}
```

内核不能直接解引用用户指针。`copyout()`

会检查虚拟地址映射和访问范围，所以官方测试传入非法地址时能够正确返回 `-1`，而不会破坏内核。

4.4 结果分析

`sysinfotest` 验证了四类行为：正常调用、非法用户地址、`sbrk()`

分配和释放页面后的空闲内存变化，以及 `fork()` 前后的进程数变化。手工执行和官方评分均输出 `sysinfotest: OK`。

5. 运行结果

5.1 xv6 内手工验证

手工启动 xv6 后依次执行 `trace 32 grep hello README` 和 `sysinfotest`。跟踪输出只包含 `read`，系统信息测试完成。

```
WSL2 | Ubuntu | xv6-labs-2021

Lab syscall - trace and sysinfo inside xv6
Actual command: make qemu

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ trace 32 grep hello README
3: syscall read -> 1023
3: syscall read -> 968
3: syscall read -> 235
3: syscall read -> 0
$ sysinfotest
sysinfotest: start
sysinfotest: OK
$ QEMU: Terminated
```

5.2 官方评分

执行 `make grade` 后，所有功能测试及 `time.txt` 检查均通过，最终得分为 35/35。

```
WSL2 | Ubuntu | xv6-labs-2021

Lab syscall - official grading result
Actual command: make grade

== Test trace 32 grep ==
$ make qemu-gdb
trace 32 grep: OK (4.9s)
== Test trace all grep ==
$ make qemu-gdb
trace all grep: OK (1.1s)
== Test trace nothing ==
$ make qemu-gdb
trace nothing: OK (1.0s)
== Test trace children ==
$ make qemu-gdb
trace children: OK (11.7s)
== Test sysinfotest ==
$ make qemu-gdb
sysinfotest: OK (1.4s)
== Test time ==
time: OK
Score: 35/35
```

6. 工具链兼容处理

2021 年代码在当前 2026 环境中遇到两类非实验逻辑问题：

- Python 3.13 及以上已删除 `pipes` 模块，因此评分器将 `pipes.quote` 等价替换为 `shlex.quote`。
- 新版 GCC 增加了递归和函数指针类型诊断；官方基础代码在 `-Werror` 下无法构建，因此只关闭 `-Winfinite-recursion` 和 `-Wincompatible-pointer-types` 两项新诊断。
- 显式指定 `-march=rv64gc`，保证新版 RISC-V 工具链使用与 xv6 2021 相符的 ISA 扩展集合。

这些修改只影响宿主机评分器和编译兼容性，不改变 `trace`、`sysinfo` 或 xv6 原有运行语义。

6.1 复现与评分验证

在 PowerShell 中进入 WSL：

```
[powershell]
wsl -d Ubuntu
```

随后执行：

```
[bash]
cd /mnt/c/Users/Aemeath/Helper/2026/OS_Design/xv6-labs-2021
git switch syscall
make clean
make grade
echo $?
```

通过标准是输出末尾出现 `Score: 35/35`，并且紧接着执行的 `echo $?` 输出

0。需要单独验证时，可执行：

```
[bash]
./grade-lab-syscall "trace 32 grep"
./grade-lab-syscall "trace all grep"
./grade-lab-syscall "trace nothing"
./grade-lab-syscall "trace children"
./grade-lab-syscall sysinfotest
```

6.2 实验总结

本实验完成了两个新系统调用及其全部注册路径。`trace` 展示了系统调用分派、进程私有状态和 `fork()` 继承；`sysinfo` 展示了内核数据结构统计、锁保护和用户/内核地址空间之间的数据复制。官方评分结果为 35/35，表明实现满足功能、隔离性、继承关系、非法地址处理和资源统计要求。

xv6 Labs 2021 实验报告（三）

Lab pgtbl: Page Tables

作者: Shalom0919

实验分支: pgtbl

本地提交: e0666e9

代码仓库: github.com/Shalom0919/xv6_labs

完成日期: 2026-07-29

官方评分: 46/46

1. 实验概述

本实验基于 MIT 6.S081 Fall 2021 的 [xv6-labs-2021](#) 仓库 [pgtbl](#) 分支，通过修改进程页表和 RISC-V 页表项，理解 Sv39 三级页表、用户和内核地址空间隔离以及硬件访问位。实验包括三个部分：

- 在 `USYSCALL` 映射只读共享页，使 `ugetpid()` 无需陷入内核。
- 实现 `vmprint()`，递归输出三级页表结构。
- 实现 `pgaccess()`，读取并清除页表项的 `PTE_A` 访问位。

官方实验说明：[MIT 6.S081 - Lab: Page tables](#)

1.1 实验环境

项目	配置
宿主环境	Windows + WSL2 Ubuntu, x86_64
目标架构	RISC-V 64, Sv39 页表
编译工具链	<code>riscv64-linux-gnu-gcc</code>
模拟器	<code>qemu-system-riscv64</code>
官方基线	<code>origin/pgtbl</code> , 提交 <code>1e6b2de</code>
实验成果	<code>pgtbl</code> 分支, 提交 <code>e0666e9</code>

1.2 Sv39 地址转换

Sv39 将虚拟地址划分为三级 9 位索引和 12 位页内偏移。`walk()` 从最高层页表开始，用 `PX(level, va)` 逐级定位页表项；非叶子 PTE 指向下一层页表，包含 `PTE_R`、`PTE_W` 或 `PTE_X` 的有效 PTE 则是最终映射。`PTE2PA()` 从页表项中提取物理页地址。

2. USYSCALL 只读共享页

2.1 进程共享页生命周期

在 `struct proc` 中保存每个进程独立的共享页：

```
[c]
struct usyscall *usyscall;
```

`allocproc()` 在 PID 分配完成后申请物理页并初始化 PID。任何分配失败都调用原有 `freeproc()` 回收已经获得的资源：

```
[c]
if((p->usyscall =
    (struct usyscall *)kalloc()) == 0){
    freeproc(p);
    release(&p->lock);
    return 0;
}
p->usyscall->pid = p->pid;
```

`freeproc()` 对应调用 `kfree()` 并将指针清零，保证进程退出和进程槽复用时不会泄漏页面。

2.2 建立只读用户映射

`proc_pagetable()` 在 USYSCALL 虚拟地址建立映射：

```
[c]
if(mappages(pagetable, USYSCALL, PGSIZE,
    (uint64)p->usyscall,
    PTE_R | PTE_U) < 0){
    uvmunmap(pagetable, TRAPFRAME, 1, 0);
    uvmunmap(pagetable, TRAMPOLINE, 1, 0);
    uvmfree(pagetable, 0);
    return 0;
}
```

权限仅包含 `PTE_R | PTE_U`：用户态可以读取，但没有 `PTE_W`，不能修改 PID。失败路径撤销已经建立的 `TRAPFRAME` 和 `TRAMPOLINE` 映射。释放页表时使用 `uvmunmap(..., 0)` 只删除映射，物理页由 `freeproc()` 统一释放。

2.3 加速原理与扩展分析

用户库中的 `ugetpid()` 直接读取固定地址：

```
[c]
struct usyscall *u =
    (struct usyscall *)USYSCALL;
return u->pid;
```

普通 `getpid()` 需要执行 `ecall`、保存用户寄存器、切换页表并进入内核分派；`ugetpid()` 只执行普通内存读取。官方测试连续创建 64 个子进程，逐一比较 `getpid()` 与 `ugetpid()`，结果为 OK，说明每个进程都映射了自己的正确 PID。

类似方法可以加速 `uptime()`：内核在共享页维护 tick 快照，用户直接读取。由于 tick 会被中断处理程序并发更新，需要使用原子值或序列计数器，避免用户观察到不一致数据。其他频繁读取、体积小且不敏感的进程属性也可采用相同方式。

3. vmprint 页表打印

3.1 递归遍历

`vmprintwalk()` 遍历每级页表的 512 个 PTE，只打印有效项。没有读、写、执行权限的有效 PTE 是非叶子节点，需要递归访问下一层：

```
[c]
static void
vmprintwalk(pagetable_t pagetable, int depth)
{
    for(int i = 0; i < 512; i++){
        pte_t pte = pagetable[i];
        if((pte & PTE_V) == 0)
            continue;

        for(int j = 0; j < depth; j++)
            printf(" ..");
        printf("%d: pte %p pa %p\n",
               i, pte, PTE2PA(pte));

        if((pte & (PTE_R | PTE_W | PTE_X)) == 0)
            vmprintwalk((pagetable_t)PTE2PA(pte),
                        depth + 1);
    }
}
```

入口函数先用 `%p` 打印根页表地址，再以深度 1 调用递归函数。`exec()` 完成新地址空间替换后，仅在 `p->pid == 1` 时调用 `vmprint(p->pagetable)`，因此启动过程只打印 `init` 的页表。

3.2 输出解释

缩进中的每个 `..` 表示深入一级页表。输出中的 `pte` 是完整页表项，`pa` 是通过 `PTE2PA()` 提取的物理地址。对 `init` 进程：

虚拟页	内容与权限
page 0	<code>init</code> 的程序代码和数据
page 1	用户栈保护页， <code>PTE_U</code> 被清除，用户态不可读写
page 2	用户栈
USYSCALL	只读 <code>struct usyscall</code> ，保存 PID
TRAPFRAME	仅内核使用的进程陷阱状态
TRAMPOLINE	用户态和内核态切换所需的 <code>trampoline</code> 代码

最高地址区域倒数第三页是 `USYSCALL`，其上依次为 `TRAPFRAME` 和 `TRAMPOLINE`。官方 `pte printout` 测试还验证了每行物理地址确实由对应 PTE 正确提取。

4. pgaccess 访问页检测

4.1 RISC-V 访问位

RISC-V 页表遍历硬件访问页面时会设置 PTE 的第 6 位，因此增加：

```
[c]
#define PTE_A (1L << 6)
```

`pgaccess(base, npages, mask)` 检查从 `base` 开始的若干页面，并把第 `i` 页的访问状态放入结果整数第 `i` 位。实现将上限设为 32 页，与 `uint32` 位图相匹配。

4.2 系统调用实现

```
[c]
uint64
sys_pgaccess(void)
{
    uint64 start, user_mask;
    int npages;
    uint32 mask = 0;
    struct proc *p = myproc();

    if(argaddr(0, &start) < 0 ||
       argint(1, &npages) < 0 ||
       argaddr(2, &user_mask) < 0)
        return -1;
    if(npages < 0 || npages > 32)
        return -1;

    for(int i = 0; i < npages; i++){
        pte_t *pte =
            walk(p->pagetable, start + i * PGSIZE, 0);
        if(pte == 0 || (*pte & PTE_V) == 0)
            return -1;
        if(*pte & PTE_A){
            mask |= 1U << i;
            *pte &= ~PTE_A;
        }
    }

    sfence_vma();
    if(copyout(p->pagetable, user_mask,
              (char *)&mask, sizeof(mask)) < 0)
        return -1;
    return 0;
}
```

结果先在内核变量中构造，最后通过 `copyout()` 安全写回用户空间。读取后清除 `PTE_A`，使下一次调用只反映此后发生的访问；`sfence_vma()` 使失效后的页表状态与后续地址转换保持一致。

4.3 结果分析

`pgtbltest` 首先调用一次 `pgaccess()` 清除旧状态，然后只访问第 1、2、30 页。第二次返回的位图必须等于 $(1 \ll 1) \mid (1 \ll 2) \mid (1 \ll 30)$ 。测试结果为 `pgaccess_test: OK`，证明位序、页间步长、访问位读取和清除均正确。

5. 运行与评分结果

5.1 xv6 内手工验证

启动 xv6 时成功输出 `init` 的三级页表；执行 `pgtbltest` 后，`ugetpid_test` 和 `pgaccess_test` 均为 OK。

WSL2 | Ubuntu | xv6-labs-2021

Lab pgtbl - page table and pgtbltest inside xv6

Actual command: `make qemu`

xv6 kernel is booting

hart 1 starting

hart 2 starting

page table 0x0000000087f6e000

..0: pte 0x0000000021fda801 pa 0x0000000087f6a000

.. ..0: pte 0x0000000021fda401 pa 0x0000000087f69000

.. ..0: pte 0x0000000021fdac1f pa 0x0000000087f6b000

.. ..1: pte 0x0000000021fda00f pa 0x0000000087f68000

.. ..2: pte 0x0000000021fd9c1f pa 0x0000000087f67000

..255: pte 0x0000000021fdb401 pa 0x0000000087f6d000

.. ..511: pte 0x0000000021fdb001 pa 0x0000000087f6c000

.. ..509: pte 0x0000000021fdd813 pa 0x0000000087f76000

.. ..510: pte 0x0000000021fddc07 pa 0x0000000087f77000

.. ..511: pte 0x0000000020001c0b pa 0x0000000080007000

init: starting sh

\$ `pgtbltest`

ugetpid_test starting

ugetpid_test: OK

pgaccess_test starting

pgaccess_test: OK

pgtbltest: all tests succeeded

\$ QEMU: Terminated

5.2 官方评分

执行 `make grade` 后，功能测试、页表格式、问答文件、完整 `usertests` 和时间文件全部通过，最终得分为 46/46。

WSL2 | Ubuntu | xv6-labs-2021

Lab pgtbl - official grading result

Actual command: `make grade`

== Test pgtbltest ==

\$ `make qemu-gdb`

(10.9s)

== Test pgtbltest: ugetpid ==

pgtbltest: ugetpid: OK

== Test pgtbltest: pgaccess ==

pgtbltest: pgaccess: OK

== Test pte printout ==

\$ `make qemu-gdb`

pte printout: OK (2.2s)

== Test answers-pgtbl.txt == answers-pgtbl.txt: OK

== Test usertests ==

\$ `make qemu-gdb`

(142.5s)

== Test usertests: all tests ==

usertests: all tests: OK

== Test time ==

time: OK

Score: 46/46

6. 兼容处理、复现与总结

6.1 2026 工具链兼容

实验逻辑之外进行了三项宿主工具兼容处理：

- Python 3.13 及以上删除了 `pipes`，评分器改用等价的 `shlex.quote`。
- 新版 GCC 对官方基础代码新增递归和函数指针类型诊断，只关闭 `-Winfinite-recursion` 与 `-Wincompatible-pointer-types` 两项诊断。
- 显式使用 `-march=rv64gc`，与 xv6 2021 使用的 RISC-V ISA 扩展相匹配。

这些修改不改变页表映射、访问位或 xv6 原有运行语义。

6.2 复现与评分验证

```
[powershell]
wsl -d Ubuntu

[bash]
cd /mnt/c/Users/Aemeath/Helper/2026/OS_Design/xv6-labs-2021
git switch pgtbl
make clean
make grade
echo $?
```

通过标准是末尾出现 `Score: 46/46`，并且 `echo $?` 输出 `0`。单独验证主要功能可执行：

```
[bash]
./grade-lab-pgtbl pgtbltest
./grade-lab-pgtbl "pte printout"
```

6.3 实验总结

本实验完成了共享只读页、页表递归打印和访问位检测。实现覆盖了物理页生命周期、页表建立与回滚、用户权限控制、三级页表递归、PTE 位操作和安全用户内存复制。官方评分 46/46 且完整 `usertests` 通过，说明新增功能未破坏 xv6 原有内存管理和进程行为。

xv6 Labs 2021 实验报告（四）

Lab traps: Traps

作者: Shalom0919

实验分支: traps

本地提交: c6baf70

代码仓库: github.com/Shalom0919/xv6_labs

完成日期: 2026-07-29

官方评分: 85/85

1. 实验概述

本实验基于 MIT 6.S081 Fall 2021 的 `xv6-labs-2021` 仓库 `traps` 分支，围绕 RISC-V 函数调用约定、内核栈帧、用户态陷阱和时钟中断展开。实验内容分为三个部分：

- 阅读 `user/call.asm`，分析参数寄存器、内联优化、返回地址和端序。
- 实现内核 `backtrace()`，沿帧指针链输出调用路径，并接入 `sys_sleep()` 与 `panic()`。
- 实现 `sigalarm()` 和 `sigreturn()`，支持按 CPU tick 周期进入用户处理函数、完整恢复中断现场并阻止处理器重入。

官方实验说明：[MIT 6.S081 - Lab: Traps](#)

1.1 实验环境

项目	配置
宿主环境	Windows + WSL2 Ubuntu, x86_64
目标架构	RISC-V 64
编译工具链	<code>riscv64-linux-gnu-gcc</code>
模拟器	<code>qemu-system-riscv64</code>
官方基线	<code>origin/traps</code> ，提交 219a8d7
实验成果	<code>traps</code> 分支，提交 c6baf70

1.2 陷阱处理路径

用户程序执行系统调用时，汇编桩把调用号放入 `a7` 并执行 `ecall`。`trampoline.S` 的 `uservec` 将用户寄存器保存到当前进程的 `trapframe`，切换到内核页表和内核栈，然后进入 `usertrap()`。系统调用由 `syscall()` 分派；时钟中断由 `devintr()` 识别并返回 2。返回用户态前，`usertrapret()` 与 `userret` 根据 `trapframe->epc` 和保存的寄存器恢复执行。

报警功能利用的正是这条路径：时钟中断到达时保存完整用户陷阱帧，并将返回地址 `epc` 改为处理函数地址；处理函数调用 `sigreturn()` 后，内核再把原陷阱帧恢复。

2. RISC-V 汇编分析

执行 `make fs.img` 会生成与当前编译器产物一致的 `user/call.asm`。本机反汇编中 `main` 的关键部分如下：

```
[asm]
0000000000000024 <main>:
2c: 4635          li    a2,13
2e: 45b1          li    a1,12
30: 00000517      auipc a0,0x0
34: 7d850513      addi  a0,a0,2008
38: 00000097      auipc ra,0x0
3c: 616080e7      jalr  1558(ra) # 64e <printf>
40: 4501          li    a0,0
```

2.1 参数、内联与返回地址

RISC-V 调用约定使用 `a0` 至 `a7` 传递前八个整数或指针参数。调用 `printf("%d %d\n", f(8)+1, 13)` 时，格式字符串、12 和 13 分别位于 `a0`、`a1` 和 `a2`，因此 13 保存在 `a2`。

`main` 中没有对 `f` 的调用，`f` 中也没有对 `g` 的调用。编译器在优化阶段内联了两个简单函数，并把 `f(8)+1` 常量折叠为 12。`printf` 位于地址 `0x64e`。`jalr` 指令位于 `0x3c`，执行后将下一条指令地址写入 `ra`，所以 `ra=0x40`。

2.2 端序与可变参数

下列程序输出 `He110 World`：

```
[c]
unsigned int i = 0x00646c72;
printf("H%x Wo%s", 57616, &i);
```

57616 的十六进制表示是 `e110`。RISC-V 为小端序，整数 `i` 在内存中排列为 `72 6c 64 00`，对应以零结尾的字符串 `rld`。若机器改为大端序，应将 `i` 改为 `0x726c6400`；57616 表示的数值不变，因此不需要修改。

对于 `printf("x=%d y=%d", 3), y` 后的结果是不确定值。格式串要求两个整数，调用者却只提供一个，`printf` 会把下一个参数寄存器中的遗留内容当作参数读取。这属于缺少可变参数造成的未定义行为，不能假定某个固定结果。

完整问答已保存到 `answers-traps.txt`。

3. 内核调用栈回溯

3.1 读取帧指针

编译选项保留帧指针。RISC-V 内核用 `s0` 保存当前帧指针，因此在 `kernel/riscv.h` 中增加 `r_fp()` 内联函数，通过 `asm volatile("mv %0, s0" : "=r" (x))` 读取当前帧指针。

GCC 生成的每个内核栈帧中，`fp-8` 保存返回地址，`fp-16` 保存调用者的帧指针。xv6 为每个进程分配一个按页对齐、大小为一页的内核栈，这为回溯提供了明确边界。

3.2 沿栈帧链遍历

`kernel/printf.c` 中的实现如下：

```
[c]
void
backtrace(void)
{
    uint64 fp = r_fp();
    uint64 stack_bottom = PGROUNDDOWN(fp);
    uint64 stack_top = PROUNDUP(fp);

    printf("backtrace:\n");
    while(fp > stack_bottom && fp < stack_top){
        uint64 return_address = *(uint64 *)(fp - 8);
        uint64 caller_fp = *(uint64 *)(fp - 16);

        printf("%p\n", return_address);
        if(caller_fp <= fp || caller_fp >= stack_top)
            break;
        fp = caller_fp;
    }
}
```

循环不仅检查当前指针位于内核栈页内，还要求调用者帧指针严格增大且不越过栈顶，避免损坏的帧链导致死循环或访问其他页面。`backtrace()` 被加入 `kernel/defs.h`，并在 `sys_sleep()` 和 `panic()` 中调用。

3.3 回溯结果分析

执行 `bttest` 得到三个地址：

```
[text]
0x00000000800021e4
0x00000000800020c0
0x0000000080001d72
```

使用 `riscv64-linux-gnu-addr2line -e kernel/kernel` 解析后，三个地址依次对应 `kernel/sysproc.c:62`、`kernel/syscall.c:144` 和 `kernel/trap.c:76`。这与 `bttest` → `sleep` → `ecall` → `usertrap` → `syscall` → `sys_sleep` 的内核调用路径相符。

4. 报警系统调用接口

4.1 用户接口与系统调用注册

在 `user/user.h` 中增加：

```
[c]
int sigalarm(int, void (*)());
int sigreturn(void);
```

`user/usys.pl` 增加同名汇编桩；`kernel/syscall.h` 分配编号 22 和 23；`kernel/syscall.c` 将编号映射到 `sys_sigalarm` 和 `sys_sigreturn`。`Makefile` 的 `traps` 用户程序列表加入 `_alarmtest`，使测试程序进入 `fs.img`。

这四层分别负责 C 语言声明、用户态 `ecall` 入口、ABI 调用号和内核分派。任何一层缺失都会造成编译、链接或运行时的未知系统调用错误。

4.2 每进程报警状态

在 `struct proc` 中保存报警配置、计数器、重入标记和完整中断现场：

```
[c]
int alarm_interval;
int alarm_ticks;
uint64 alarm_handler;
int alarm_active;
struct trapframe alarm_trapframe;
```

`allocproc()` 初始化所有字段，`freeproc()`

再次清零，防止进程槽复用时空留旧报警状态。使用完整 `struct trapframe` 而不是只保存 `epc`，是因为被中断程序的通用寄存器、栈指针、返回地址和参数寄存器都可能处于有效计算过程中。

4.3 配置报警

`sys_sigalarm()` 读取间隔和用户函数地址，拒绝负间隔，并重置当前计数：

```
[c]
uint64
sys_sigalarm(void)
{
    int interval;
    uint64 handler;
    struct proc *p = myproc();

    if(argint(0, &interval) < 0 ||
        argaddr(1, &handler) < 0)
        return -1;
    if(interval < 0)
        return -1;

    p->alarm_interval = interval;
    p->alarm_handler = handler;
    p->alarm_ticks = 0;
    return 0;
}
```

`sigalarm(0, 0)` 通过把间隔设为 0 停止后续报警。判断报警是否启用时只检查间隔，不检查处理函数地址，因为用户函数完全可能被链接到地址 0，官方测试中的 `periodic` 就可能出现这种情况。

5. 报警触发与现场恢复

5.1 时钟中断触发用户处理函数

在 `usertrap()` 确认进程未被杀死后，仅对用户态时钟中断计数：

```
[c]
if(which_dev == 2 &&
    p->alarm_interval > 0 &&
    !p->alarm_active){
    p->alarm_ticks++;
    if(p->alarm_ticks >= p->alarm_interval){
        memmove(&p->alarm_trapframe, p->trapframe,
                sizeof(p->alarm_trapframe));
        p->alarm_ticks = 0;
        p->alarm_active = 1;
        p->trapframe->epc = p->alarm_handler;
    }
}
```

到达间隔时，内核先复制完整陷阱帧，再把 `alarm_active` 置位，最后把返回用户态的 `epc` 改为处理函数地址。`usertrapret()` 随后按正常陷阱返回流程进入处理函数，不需要伪造内核调用栈。

`alarm_active` 是防止重入的关键。如果处理函数本身执行时间超过一个报警周期，后续时钟中断不会再次覆盖保存现场，也不会递归进入处理函数。官方 `test2` 专门验证这一条件。

5.2 sigreturn 恢复完整现场

用户处理函数结束前主动调用 `sigreturn()`：

```
[c]
uint64
sys_sigreturn(void)
{
    struct proc *p = myproc();
    uint64 interrupted_a0;

    if(!p->alarm_active)
        return -1;

    interrupted_a0 = p->alarm_trapframe.a0;
    memmove(p->trapframe, &p->alarm_trapframe,
            sizeof(p->alarm_trapframe));
    p->alarm_active = 0;
    p->alarm_ticks = 0;
    return interrupted_a0;
}
```

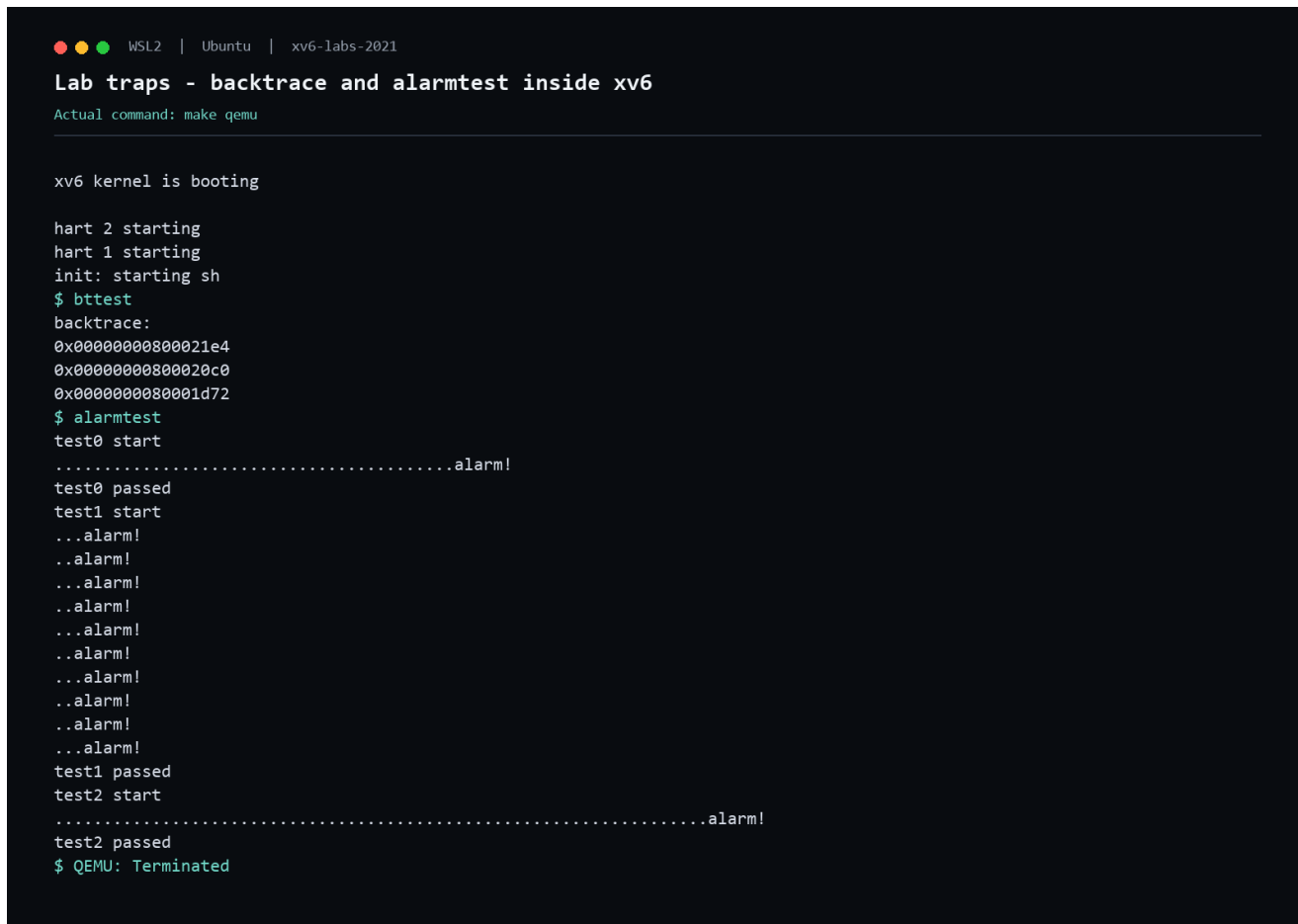
恢复完整陷阱帧后，系统调用分派器仍会把 `sys_sigreturn()` 的返回值写入当前 `trapframe->a0`。因此实现先取出被中断时的 `a0` 并返回它，使分派器写回的仍是原值。若固定返回 0，循环变量或临时计算恰好位于 `a0` 时就会被破坏，`alarmtest` 的寄存器一致性检查可能失败。

恢复完成后清除重入标记并将计数归零，下一周期从头计数。若处理函数内部调用 `sigalarm(0, 0)`，间隔会保持为 0，`sigreturn()` 恢复现场后也不会再次触发。

6. 实验结果与分析

6.1 xv6 内交互验证

启动 xv6 后依次执行 `bttest` 和 `alarmtest`。回溯输出三层有效内核调用路径；报警测试中，`test0` 验证至少触发一次，`test1` 验证周期触发和寄存器恢复，`test2` 验证长处理函数不会重入，三项均通过。



```
WSL2 | Ubuntu | xv6-labs-2021

Lab traps - backtrace and alarmtest inside xv6
Actual command: make qemu

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ bttest
backtrace:
0x00000000800021e4
0x00000000800020c0
0x0000000080001d72
$ alarmtest
test0 start
.....alarm!
test0 passed
test1 start
...alarm!
..alarm!
...alarm!
..alarm!
..alarm!
..alarm!
..alarm!
..alarm!
..alarm!
..alarm!
..alarm!
test1 passed
test2 start
.....alarm!
test2 passed
$ QEMU: Terminated
```

图 1 `bttest` 与 `alarmtest` 在 `xv6` 中的实际运行结果

6.2 官方评分

执行官方 `make grade` 后，汇编问答、`backtrace`、`alarmtest test0/test1/test2`、完整 `usertests` 和时间文件全部通过，最终得分为 85/85。

```
WSL2 | Ubuntu | xv6-labs-2021

Lab traps - official grading result

Actual command: make grade

make: 'kernel/kernel' is up to date.
== Test answers-traps.txt == answers-traps.txt: OK
== Test backtrace test ==
$ make qemu-gdb
backtrace test: OK (2.0s)
== Test running alarmtest ==
$ make qemu-gdb
(3.2s)
== Test alarmtest: test0 ==
alarmtest: test0: OK
== Test alarmtest: test1 ==
alarmtest: test1: OK
== Test alarmtest: test2 ==
alarmtest: test2: OK
== Test usertests ==
$ make qemu-gdb
usertests: OK (148.9s)
== Test time ==
time: OK
Score: 85/85
```

图 2 官方评分脚本输出

usertests: OK

表明新增陷阱逻辑没有破坏原有系统调用、进程调度、文件系统和虚拟内存行为。官方 backtrace 测试还将三个地址逐一交给 `addr2line`，因此通过不只是输出格式正确，也验证了调用链内容。

7. 兼容处理与复现验证

7.1 2026 工具链兼容

官方 2021 基线在当前工具链上需要三项兼容处理：

- Python 新版本移除了 `pipes`，评分器改用等价的 `shlex.quote`。
- 新版 GCC 对官方旧代码新增递归和函数指针类型诊断，只关闭 `-Winfinite-recursion` 与 `-Wincompatible-pointer-types` 两项诊断。
- 显式使用 `-march=rv64gc`，与 xv6 所需的 RISC-V 指令集扩展匹配。

这些修改只解决宿主工具版本差异，不改变 `trap`、系统调用或报警功能的语义。

7.2 官方评分复现

在 Windows PowerShell 中进入 WSL：

```
[powershell]
wsl -d Ubuntu
```

然后运行：

```
[bash]
cd /mnt/c/Users/Aemeath/Helper/2026/OS_Design/xv6-labs-2021
git switch traps
make clean
make grade
echo $?
```

通过标准是输出末尾出现 `Score: 85/85`，且 `echo $?` 输出 0。也可以单独验证各部分：

```
[bash]
./grade-lab-traps "backtrace test"
./grade-lab-traps "alarmtest: test0"
./grade-lab-traps "alarmtest: test1"
./grade-lab-traps "alarmtest: test2"
./grade-lab-traps usertests
```

现场演示时可执行：

```
[bash]
make qemu
```

进入 xv6 shell 后输入 `bttest` 和 `alarmtest`；退出 QEMU 使用 `Ctrl-a`，再按 `x`。

8. 实验总结

本实验完成了 RISC-V 汇编问答、内核栈回溯以及用户级周期报警。实现覆盖了函数调用约定、帧指针链、陷阱帧、时钟中断、系统调用 ABI、完整寄存器现场保存与恢复，以及用户处理函数的重入控制。

`backtrace()` 的实质是用 ABI 约定解释内核栈；报警机制的实质则是让内核临时修改陷阱返回现场，把一次普通的中断返回转换为用户处理函数调用，再由 `sigreturn()` 恢复原计算。官方评分 85/85 且完整 `usertests` 通过，说明功能实现和原系统兼容性均达到实验要求。

xv6 Labs 2021 实验报告（五）

Lab cow: Copy-on-Write Fork

作者: Shalom0919

实验分支: cow

本地提交: a419e42

代码仓库: github.com/Shalom0919/xv6_labs

完成日期: 2026-07-30

官方评分: 110/110

1. 实验概述

本实验基于 MIT 6.S081 Fall 2021 的 [xv6-labs-2021](#) 仓库 `cow` 分支，实现 Copy-on-Write Fork。原始 `xv6` 的 `fork()` 会立即为子进程分配物理页并复制父进程全部用户内存；当进程较大，或子进程随后立即执行 `exec()` 时，这些复制会消耗大量时间和物理内存。

COW fork 将复制推迟到实际写入时：父子页表先共同指向相同物理页，并清除可写权限。任一进程写入共享页时，RISC-V 产生 `store page fault`，内核再为写入者建立私有副本。实验实现包括：

- 修改 `uvmcopy()`，建立父子共享的只读 COW 映射。
- 使用 RISC-V PTE 的 `RSW` 位区分 COW 页与真正只读页。
- 为每个可分配物理页维护并发安全的引用计数。
- 在 `usertrap()` 中处理 COW 写故障。
- 让内核 `copyout()` 写用户空间时执行相同的 COW 拆分。

官方实验说明：[MIT 6.S081 - Lab: Copy-on-Write Fork](#)

1.1 实验环境

项目	配置
宿主环境	Windows + WSL2 Ubuntu, x86_64
目标架构	RISC-V 64, Sv39
编译工具链	<code>riscv64-linux-gnu-gcc</code>
模拟器	<code>qemu-system-riscv64</code>
官方基线	<code>origin/cow</code> , 提交 <code>c981891</code>
实验成果	<code>cow</code> 分支, 提交 <code>a419e42</code>

1.2 COW 状态转换

事件	页表权限与引用计数
<code>kalloc()</code>	新物理页引用计数设为 1
<code>fork()</code>	原可写页清除 <code>PTE_W</code> 、设置 <code>PTE_COW</code> ，子映射引用计数加 1
多引用页被写	分配新页、复制内容、写入者改映射，旧页引用计数减 1
单引用 COW 页被写	不复制，只恢复 <code>PTE_W</code> 并清除 <code>PTE_COW</code>
进程退出或缩小内存	删除映射并递减计数；计数归零才进入空闲链表

真正的代码页原本没有 `PTE_W`。`fork()` 不应把这类页面标记成 COW，否则对只读代码的非法写入会被错误地转化为合法私有副本。

2. COW 页表标记

2.1 使用 RSW 软件保留位

Sv39 PTE 的第 8、9 位为 RSW，硬件不会解释，可由操作系统保存软件状态。本实验使用第 8 位：

```
[c]
#define PTE_COW (1L << 8)
```

页表项的关键状态为：

- 普通可写页：PTE_V | PTE_U | PTE_W。
- COW 共享页：保留原读、执行和用户权限，清除 PTE_W，设置 PTE_COW。
- 真正只读页：没有 PTE_W，也没有 PTE_COW。

写入 COW 页会触发异常；写入真正只读页同样触发异常，但后者不满足 PTE_COW 条件，进程应被终止。

2.2 uvmcopy 建立共享映射

原始 uvmcopy() 对每一页执行 kalloc() 和 memmove()。修改后，父子直接共享物理地址：

```
[c]
for(i = 0; i < sz; i += PGSIZE){
    pte = walk(old, i, 0);
    pa = PTE2PA(*pte);
    flags = PTE_FLAGS(*pte);
    if(flags & PTE_W){
        flags = (flags & ~PTE_W) | PTE_COW;
        *pte = PA2PTE(pa) | flags;
    }
    if(mappages(new, i, PGSIZE, pa, flags) != 0)
        goto err;
    krefinc(pa);
}
sfence_vma();
```

只对原可写页设置 COW。已经处于 COW 状态的页会保留该标记，使连续多次 fork() 可以继续共享。子页表建立映射成功后增加物理页引用计数。

修改父页表的 PTE_W 后执行 sfence_vma()，清除可能仍允许写入的旧 TLB 项。否则父进程返回用户态后可能绕过新的只读权限，直接修改共享页。

2.3 失败回滚

若建立子页表映射失败，uvmunmap(new, 0, i / PGSIZE, 1) 删除已完成的子映射。每次 kfree() 会递减引用计数，因此共享页不会被提前释放。

部分父页可能已经变为 COW，但此时引用计数仍为

1。父进程下次写入时走单引用快速路径，直接恢复可写权限，不需要额外复制。这保证 fork() 失败后父进程仍可继续运行。

3. 物理页引用计数

3.1 索引和并发保护

一张物理页可能同时被多个进程页表引用，只有最后一个映射消失时才能释放。本实验按物理页号建立固定数组：

```
[c]
#define NPHYPAGES ((PHYSTOP - KERNBASE) / PGSIZE)
#define PAINDEX(pa) (((uint64)(pa) - KERNBASE) / PGSIZE)

struct {
    struct spinlock lock;
    int count[NPHYPAGES];
} krefs;
```

数组仅覆盖 QEMU 提供给 xv6

的物理内存区间，避免按完整物理地址建立过大的稀疏数组。所有增加、减少和读取操作都受 `krefs.lock` 保护，防止多个 CPU 同时 fork、退出或处理写故障时丢失更新。

3.2 分配和释放语义

`kalloc()` 从空闲链表取出页面后，将引用计数从 0 设置为 1：

```
[c]
if(r){
    acquire(&krefs.lock);
    if(krefs.count[PAINDEX(r)] != 0)
        panic("kalloc ref");
    krefs.count[PAINDEX(r)] = 1;
    release(&krefs.lock);
    memset((char*)r, 5, PGSIZE);
}
```

`kfree()` 不再无条件释放。它先递减引用计数，只有计数归零才填充调试字节并加入空闲链表：

```
[c]
acquire(&krefs.lock);
if(krefs.count[PAINDEX(pa)] < 1){
    release(&krefs.lock);
    panic("kfree ref");
}
count = --krefs.count[PAINDEX(pa)];
release(&krefs.lock);

if(count > 0)
    return;

memset(pa, 1, PGSIZE);
/* add pa to kmem.freelist */
```

启动阶段的空闲页尚未经过 `kalloc()`。`freerange()` 在首次调用 `kfree()` 前给每页增加一个临时引用，使其由 1 正常递减到 0，再进入空闲链表。页表页、内核栈和普通用户页仍遵循原有“一次分配、一次释放”的计数路径。

4. COW 写故障处理

4.1 统一 `cowalloc`

用户态写故障和内核 `copyout()` 都需要拆分 COW 页，因此将核心逻辑集中在 `cowalloc()`:

```
[c]
va = PGROUNDDOWN(va);
pte = walk(pagetable, va, 0);
if(pte == 0 || (*pte & PTE_V) == 0 ||
    (*pte & PTE_U) == 0 ||
    (*pte & PTE_COW) == 0)
    return -1;
pa = PTE2PA(*pte);
flags = (PTE_FLAGS(*pte) | PTE_W) & ~PTE_COW;
if(krefcnt(pa) == 1){
    *pte = PA2PTE(pa) | flags;
    sfence_vma();
    return 0;
}
if((mem = kalloc()) == 0)
    return -1;
memmove(mem, (char*)pa, PGSIZE);
*pte = PA2PTE(mem) | flags;
kfree((void*)pa);
sfence_vma();
```

当旧页引用计数为 1 时，没有其他页表能观察到写入，可以原地恢复可写权限。引用数大于 1 时才分配和复制，随后当前页表指向新页，并通过 `kfree(old_pa)` 递减旧页计数。

`kalloc()` 失败时返回错误，用户写故障路径会杀死进程；原 PTE 和旧引用计数均保持不变。

4.2 `usertrap` 识别 `store page fault`

RISC-V 的 `store/AMO page fault` 原因号为 15。在系统调用和设备中断分支之间增加：

```
[c]
} else if(r_scause() == 15){
    uint64 va = r_stval();
    if(va >= p->sz ||
        cowalloc(p->pagetable, va) < 0)
        p->killed = 1;
}
```

`r_stval()` 提供发生故障的用户虚拟地址。实现同时检查进程大小、PTE 有效位、用户位和 COW 位。写入越界地址、栈保护页或真正只读页都会失败并终止进程。

处理成功后 `usertrapret()` 回到产生故障的原指令。此时 PTE 已指向当前进程的可写页，CPU 重新执行写指令即可完成操作。

5. 内核 copyout 路径

5.1 为什么硬件不会自动触发

`copyout()` 在内核态通过物理地址直接写用户页。内核使用自己的直接映射，不会以用户 PTE 的只读权限执行这次写入，因此不会产生用户态 store page fault。如果直接写共享页，父子进程会同时看到修改，破坏 COW 隔离。

每次跨页复制前先检查目标 PTE：

```
[c]
va0 = PGROUNDDOWN(dstva);
if(va0 >= MAXVA)
    return -1;

pte = walk(pagetable, va0, 0);
if(pte != 0 && (*pte & PTE_COW) != 0){
    if(cowalloc(pagetable, va0) < 0)
        return -1;
}

pa0 = walkaddr(pagetable, va0);
if(pa0 == 0)
    return -1;
```

拆分完成后重新调用 `walkaddr()`，取得新物理页地址，再执行原有 `memmove()`。这样管道读取、文件读取和系统调用结果写回用户缓冲区都遵循 COW 语义。

5.2 非法地址边界分析

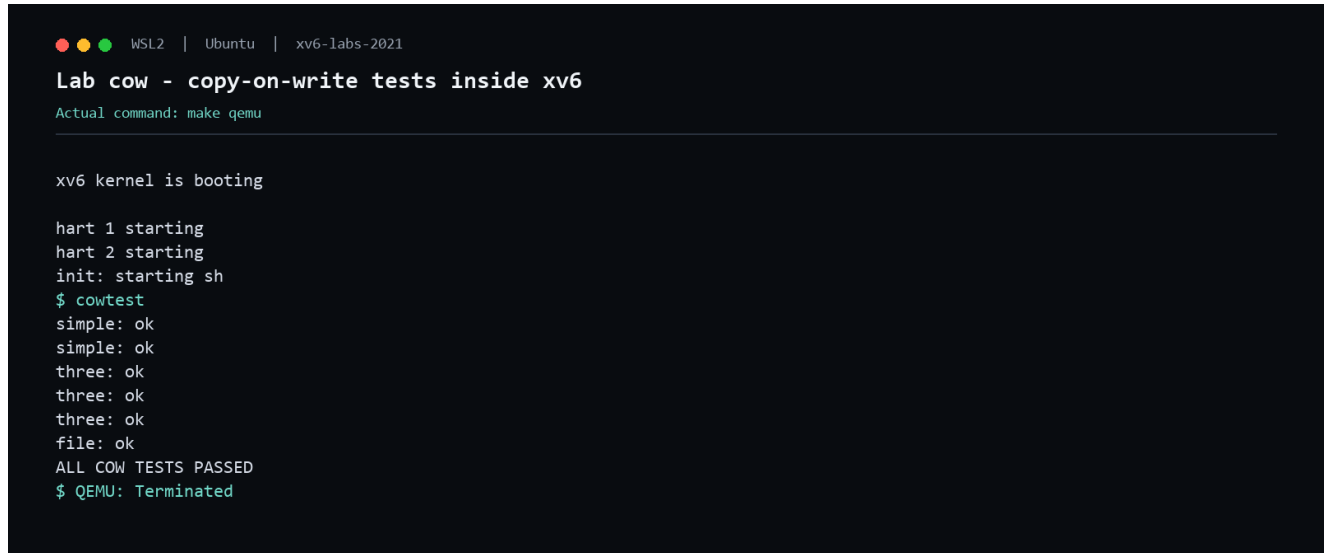
首次完整评分中，`cowtest` 全部通过，但 `usertests` 的 `copyout` 非法地址测试触发 panic: walk。原因是新增代码在原 `walkaddr()` 之前直接调用 `walk()`；而 `walk()` 对 `va >= MAXVA` 会 panic，原有 `walkaddr()` 则会安全返回 0。

最终实现先检查 `va0 >= MAXVA` 并返回 -1，保留原接口的错误语义。修复后 `usertests: copyin`、`usertests: copyout` 和所有回归测试全部通过。这个问题说明在插入新页表访问时，必须保持原函数的输入校验顺序和失败行为。

6. 实验结果与分析

6.1 cowtest 交互验证

在 xv6 shell 中运行 `cowtest`:



```
WSL2 | Ubuntu | xv6-labs-2021
Lab cow - copy-on-write tests inside xv6
Actual command: make qemu

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ cowtest
simple: ok
simple: ok
three: ok
three: ok
three: ok
file: ok
ALL COW TESTS PASSED
$ QEMU: Terminated
```

图 1 `cowtest` 在 `xv6` 中的实际输出

测试结果分析:

- `simple` 连续两次通过: 进程分配超过一半物理内存后仍能 `fork`, 说明 `fork` 不再立即复制全部页面; 测试重复执行也说明页面最终得到释放。
- `three` 连续三次通过: 三个进程对不同范围的共享页写入后内容互不干扰, 且多引用页的复制和释放正确。
- `file: ok`: 内核通过管道向子进程缓冲区执行 `copyout()` 时, 正确拆分了 COW 页, 父进程缓冲区未被覆盖。
- `ALL COW TESTS PASSED`: 内存压力、隔离性和回收路径全部满足专项测试。

6.2 官方评分

执行 `make grade` 后, 所有评分项通过, 最终得分为 110/110:

```
WSL2 | Ubuntu | xv6-labs-2021

Lab cow - official grading result

Actual command: make grade

make: 'kernel/kernel' is up to date.
== Test running cowtest ==
$ make qemu-gdb
(4.1s)
== Test simple ==
  simple: OK
== Test three ==
  three: OK
== Test file ==
  file: OK
== Test usertests ==
$ make qemu-gdb
(151.1s)
== Test usertests: copyin ==
  usertests: copyin: OK
== Test usertests: copyout ==
  usertests: copyout: OK
== Test usertests: all tests ==
  usertests: all tests: OK
== Test time ==
  time: OK
Score: 110/110
```

图 2 官方评分脚本输出

完整 `usertests` 通过说明 COW 修改没有破坏进程创建、退出、内存增长和收缩、系统调用参数复制、文件系统或其他既有内核行为。

7. 兼容处理与复现验证

7.1 2026 工具链兼容

官方 2021 基线在当前环境中进行了以下宿主兼容处理：

- Python 新版本移除了 `pipes`，评分器改用等价的 `shlex.quote`。
- 新版 GCC 对旧版 xv6 增加递归和函数指针类型诊断，只关闭对应的两项诊断。
- 显式使用 `-march=rv64gc`，与 xv6 使用的 RISC-V 指令集扩展匹配。

这些修改不改变 COW、页表或物理页分配语义。

7.2 官方评分复现

在 PowerShell 中进入 WSL：

```
[powershell]
wsl -d Ubuntu
```

然后运行：

```
[bash]
cd /mnt/c/Users/Aemeath/Helper/2026/OS_Design/xv6-labs-2021
git switch cow
make clean
make grade
echo $?
```

通过标准是末尾出现 **Score: 110/110**，且 `echo $?` 输出 0。单独验证主要评分项可以执行：

```
[bash]
./grade-lab-cow simple
./grade-lab-cow three
./grade-lab-cow file
./grade-lab-cow "usertests: copyout"
./grade-lab-cow "usertests: all tests"
```

现场运行：

```
[bash]
make qemu
```

进入 xv6 shell 后输入 `cowtest`。退出 QEMU 使用 `Ctrl-a`，再按 `x`。

8. 实验总结

本实验将 xv6 的 eager fork 改为写时复制 fork。父子页表在 fork 时共享物理页，PTE 软件位记录 COW 状态，硬件只读保护把第一次写入转换为内核可处理的 page fault；引用计数解决共享页生命周期问题，`cowalloc()` 统一用户写故障与内核 `copyout()` 的拆分语义。

实现同时处理了真正只读页、连续 fork、多进程写入、内存不足、最后引用快速路径、TLB 刷新和非法虚拟地址。官方评分 110/110 且完整 `usertests` 通过，说明 COW 功能和原系统兼容性均达到实验要求。

xv6 Labs 2021 实验报告（六）

Lab thread: [Multithreading](#)

作者: Shalom0919

实验分支: thread

本地提交: 361efe4

代码仓库: github.com/Shalom0919/xv6_labs

完成日期: 2026-08-17

官方评分: 60/60

1. 实验概述

本实验基于 MIT 6.S081 Fall 2021 的 [xv6-labs-2021](#) 仓库 `thread` 分支，围绕用户级线程与 POSIX 多线程同步完成三个任务：实现 RISC-V 用户线程上下文切换；修复并行哈希表的竞态并保持并行性能；使用互斥锁和条件变量实现可重复使用的 `barrier`。

三个任务分别体现了并发系统中的三类核心问题：

- **uthread**：线程切换时，哪些寄存器状态必须跨函数调用保存和恢复。
- **ph**：多个线程更新共享链表时，如何避免 `lost update`，并缩小锁粒度保留并行性。
- **barrier**：如何用条件变量让一轮中的所有线程会合，同时正确区分连续的多轮同步。

官方实验说明：[MIT 6.S081 - Lab: Multithreading](#)

1.1 实验环境

项目	配置
宿主环境	Windows + WSL2 Ubuntu, x86_64
目标架构	RISC-V 64, xv6 运行于 QEMU
编译工具链	<code>riscv64-linux-gnu-gcc</code> 与宿主 <code>gcc -pthread</code>
官方基线	<code>origin/thread</code> , 提交 <code>7e0a45c</code>
实验成果	<code>thread</code> 分支, 提交 <code>361efe4</code>
自动评分	<code>make grade</code> , 60/60

1.2 修改文件

文件	主要作用
<code>user/uthread.c</code>	定义线程上下文，初始化新线程栈和入口，调用切换函数
<code>user/uthread_switch.S</code>	保存旧线程、恢复新线程的 <code>callee-saved</code> 寄存器
<code>notxv6/ph.c</code>	为每个哈希桶增加互斥锁
<code>notxv6/barrier.c</code>	实现带轮次号的可重用 <code>barrier</code>
<code>answers-thread.txt</code>	分析并行 <code>put()</code> 丢键的竞态原因
<code>time.txt</code>	记录实验用时
<code>Makefile</code> 、 <code>gradelib.py</code>	适配当前 RISC-V 工具链和 Python 版本

2. 用户级线程上下文切换

2.1 上下文结构设计

每个线程拥有独立栈、运行状态和保存的寄存器上下文：

```
[c]
struct thread_context {
    uint64 ra;
    uint64 sp;
    uint64 s0;
    uint64 s1;
    /* s2 ... s9 */
    uint64 s10;
    uint64 s11;
};

struct thread {
    char stack[STACK_SIZE];
    int state;
    struct thread_context context;
};
```

根据 RISC-V 调用约定，`ra`、`sp` 和 `s0~s11` 必须在切换后保持。`a0~a7`、`t0~t6` 属于 caller-saved 寄存器，调用 `thread_switch()` 的 C 代码不能假设它们在函数返回后仍保留，所以不需要作为线程持久上下文保存。

`thread_switch(old, new)` 被普通 C 函数调用。调用本身把返回地址写入 `ra`，因此保存旧 `ra` 就等价于记录旧线程恢复后应继续执行的位置；恢复新 `ra` 后执行 `ret`，控制流便进入新线程上次暂停的位置。

2.2 新线程初始化

新线程从未执行过 `thread_switch()`，需要人工构造第一次恢复所需的最小上下文：

```
[c]
memset(&t->context, 0, sizeof(t->context));
t->context.ra = (uint64)func;
t->context.sp = ((uint64)t->stack + STACK_SIZE) & ~15;
t->state = RUNNABLE;
```

栈从数组高地址向低地址增长，初始 `sp` 指向栈顶，并按 RISC-V ABI 要求做 16 字节对齐。初始 `ra` 设置为线程入口函数；调度器第一次恢复该线程后，汇编中的 `ret` 直接跳转到 `func`。

实现还检查线程槽是否耗尽，避免越过 `all_thread` 数组后写坏内存。

2.3 调度器衔接

调度器找到下一个 `RUNNABLE` 线程后更新状态和全局当前线程指针，再切换上下文：

```
[c]
next_thread->state = RUNNING;
t = current_thread;
current_thread = next_thread;
thread_switch(&t->context, &next_thread->context);
```

这里必须先更新 `current_thread`。切换后 CPU 已在新线程上运行，新线程调用 `thread_yield()` 时应当看到自己的线程控制块。旧线程以后被重新调度时，`thread_switch()` 才从原调用点返回。

3. RISC-V 汇编切换实现

`user/uthread_switch.S` 中的实现只依赖两个参数：`a0` 指向旧上下文，`a1` 指向新上下文。寄存器以固定偏移顺序保存：

```
[asm]
thread_switch:
    sd ra,    0(a0)
    sd sp,    8(a0)
    sd s0,   16(a0)
    sd s1,   24(a0)
    sd s2,   32(a0)
    sd s3,   40(a0)
    sd s4,   48(a0)
    sd s5,   56(a0)
    sd s6,   64(a0)
    sd s7,   72(a0)
    sd s8,   80(a0)
    sd s9,   88(a0)
    sd s10,  96(a0)
    sd s11, 104(a0)

    ld ra,    0(a1)
    ld sp,    8(a1)
    ld s0,   16(a1)
    /*      s1   s11 */
    ret
```

保存和恢复顺序必须与 C 结构体字段布局完全一致，每个 `uint64` 占 8 字节。切换 `sp` 后，后续执行已经使用新线程栈；最后 `ret` 使用刚恢复的 `ra`，完成控制流转移。

实验程序中 A、B、C 三个线程每轮打印一次并主动 `yield`。实际输出始终按 C、A、B 循环，到第 99 轮后依次退出，说明程序计数位置、栈以及被调用者保存寄存器均能跨 300 余次切换保持正确。

4. 并行哈希表

4.1 丢键竞态分析

原始 `put()` 以单链表头插法向桶中加入节点。若两个不同的 `key` 映射到同一桶，两个线程可能同时读取旧的 `table[i]`，分别令新节点的 `next` 指向同一个旧表头，之后又先后写 `table[i]`。最后一次表头写入覆盖前一次写入，其中一个新节点从链表中不可达，因此测试报告 `key missing`。

该问题不是内存分配失败，也不是不同 `key` 被当成相同 `key`；本质是“读旧表头—构造新节点—写新表头”这一复合操作缺少互斥，产生 `lost update`。分析记录写入 `answers-thread.txt`。

4.2 每桶锁方案

使用 `NBUCKET` 个互斥锁，而不是一把全局锁：

```
[c]
struct entry *table[NBUCKET];
pthread_mutex_t bucket_lock[NBUCKET];

void put(int key, int value)
{
    int i = key % NBUCKET;
    pthread_mutex_lock(&bucket_lock[i]);
    /*
     *
     * ...
     */
    pthread_mutex_unlock(&bucket_lock[i]);
}
```

锁覆盖查找、更新或插入的完整临界区，确保同一桶内的链表结构串行修改。不同桶之间没有共享链表节点，因而仍可并行操作。`get()` 也使用对应桶锁完成遍历；当前程序不删除节点，返回的节点在解锁后仍有效。

所有 `bucket mutex` 在创建工作线程之前初始化。这样既消除数据竞争，又不会引入锁初始化与并发访问之间的竞态。

4.3 性能结果

手工测试使用相同的 100000 次插入工作量：

模式	时间	吞吐量	丢失 key
<code>./ph 1</code>	5.336 s	18,742 puts/s	0
<code>./ph 2</code>	2.957 s	33,816 puts/s	0

两线程相对单线程的插入吞吐提升为 $33816 / 18742 \approx 1.80$ 倍，高于官方 `ph_fast` 要求的 1.25 倍。结果表明每桶锁让发生哈希冲突的操作互斥，同时保留了不同桶之间的大部分并行度。

5. 可重用 Barrier

5.1 同步状态与实现

`barrier` 需要让每一轮的所有线程到齐后同时继续，并重复执行多轮。共享状态包括已到达线程数 `nthread`、轮次号 `round`、互斥锁和条件变量。初始化时显式把计数与轮次设为 0。

```
[c]
static void barrier(void)
{
    int current_round;

    pthread_mutex_lock(&bstate.barrier_mutex);
    current_round = bstate.round;
    bstate.nthread++;

    if(bstate.nthread == nthread){
        bstate.nthread = 0;
        bstate.round++;
        pthread_cond_broadcast(&bstate.barrier_cond);
    } else {
        while(current_round == bstate.round)
            pthread_cond_wait(&bstate.barrier_cond,
                             &bstate.barrier_mutex);
    }
    pthread_mutex_unlock(&bstate.barrier_mutex);
}
```

最后到达的线程把到达数清零、推进轮次并广播唤醒所有等待者。其他线程在 `pthread_cond_wait()` 中原子释放 `mutex` 并睡眠，醒来后重新获得 `mutex`。

5.2 为什么必须使用 `round` 和 `while`

只检查 `nthread` 不足以区分相邻两轮：某个快线程可能进入下一轮并修改计数，而上一轮的慢线程才刚被唤醒。每个等待者保存进入 `barrier` 时的 `current_round`，只有全局轮次发生变化才允许离开，因此唤醒属于哪一轮是明确的。

条件等待必须放在 `while` 中而不是 `if` 中。POSIX 条件变量允许虚假唤醒；同时，被广播唤醒的线程重新竞争 `mutex` 后也需要再次确认谓词。`while(current_round == bstate.round)` 使正确性建立在受锁保护的状态谓词上，而不是建立在“收到一次信号”上。

广播必须在持锁状态下更新轮次之后执行，保证醒来的线程重新加锁时一定观察到新的 `round`。

6. 实验结果

6.1 手工运行验证

uthread 在 xv6 中完成全部线程切换；宿主机上的 ph 单线程和双线程测试均无丢键，barrier 2 输出 OK; passed:

```
WSL2 | Ubuntu | xv6-labs-2021

Lab thread - user threads, hash table and barrier
Actual command: make qemu; ./ph 1; ./ph 2; ./barrier 2

$ uthread
thread_a started
thread_b started
thread_c started
thread_c 0
thread_a 0
thread_b 0
thread_c 99
thread_a 99
thread_b 99
thread_c: exit after 100
thread_a: exit after 100
thread_b: exit after 100
thread_schedule: no runnable threads

$ ./ph 1; ./ph 2; ./barrier 2
100000 puts, 5.336 seconds, 18742 puts/second
0: 0 keys missing
100000 puts, 2.957 seconds, 33816 puts/second
1: 0 keys missing
0: 0 keys missing
OK; passed
```

图 1 用户线程、并行哈希表与 barrier 的实际运行结果

6.2 官方评分

在 thread 分支执行 `make clean && make`

grade, 评分脚本依次检查用户线程输出、竞态分析文本、哈希表安全性、两线程性能、barrier 和用时记录:

```
WSL2 | Ubuntu | xv6-labs-2021

Lab thread - official grading result
Actual command: make grade

== Test uthread ==
uthread: OK (5.6s)
== Test answers-thread.txt == answers-thread.txt: OK
== Test ph_safe == make[1]: Entering directory '/mnt/c/Users/Aemeath/Helper/2026/OS_Design/xv6-labs-2021'
ph_safe: OK (9.1s)
== Test ph_fast == make[1]: Entering directory '/mnt/c/Users/Aemeath/Helper/2026/OS_Design/xv6-labs-2021'
ph_fast: OK (19.0s)
== Test barrier == make[1]: Entering directory '/mnt/c/Users/Aemeath/Helper/2026/OS_Design/xv6-labs-2021'
barrier: OK (3.3s)
== Test time ==
time: OK
Score: 60/60
```

图 2 官方 grade-lab-thread 评分结果

各项得分为: uthread 20/20、竞态分析 5/5、ph_safe 10/10、ph_fast 10/10、barrier 14/14、time 1/1, 合计 60/60。

7. 兼容性与复现方法

当前较新的 RISC-V 工具链会对实验基线代码产生递归与指针类型相关警告，并可能默认生成 xv6 链接脚本不接受的指令扩展属性。本实验在 `Makefile` 中补充兼容性选项和明确的 `rv64gc` 架构参数；评分库中把 Python 3 已移除的 `pipes.quote` 替换为等价的 `shlex.quote`。这些修改不改变实验算法或评分逻辑。

复现官方评分：

```
[sh]
git switch thread
make clean
make grade
echo $?
```

通过标准是最后输出 `Score: 60/60`，且退出码为 0。单独复核三个任务可执行：

```
[sh]
make qemu      # xv6 uthread
make ph barrier
./ph 1
./ph 2
./barrier 2
```

8. 实验总结

本实验完成了从寄存器级上下文切换到共享数据互斥、再到多轮条件同步的完整并发实践。用户线程部分说明上下文切换的本质是保存可恢复的控制流与 ABI 状态；哈希表部分说明锁的正确粒度既决定安全性，也决定并行性能；`barrier` 部分则说明条件变量必须围绕受锁保护的状态谓词使用，并用 `generation/round` 区分多轮事件。最终实现通过官方全部测试，得分 60/60。

xv6 Labs 2021 实验报告（七）

Lab net: Network Driver

作者: Shalom0919

实验分支: net

本地提交: a611d08

代码仓库: github.com/Shalom0919/xv6_labs

完成日期: 2026-08-17

官方评分: 100/100

1. 实验概述

本实验基于 MIT 6.S081 Fall 2021 的 [xv6-labs-2021](#) 仓库 `net` 分支，为 QEMU 模拟的 Intel E1000 网卡补全发送与接收驱动。xv6 已提供 PCI 发现、E1000 初始化、mbuf 管理以及 Ethernet、ARP、IP、UDP、DNS 协议处理；实验任务集中在 `kernel/e1000.c` 的 `e1000_transmit()` 和 `e1000_recv()`。

驱动通过内存映射寄存器与网卡交互，通过 DMA 描述符环传递数据：

- 发送路径把 mbuf 数据地址写入 TX descriptor，并推进 `E1000_TDT` 通知设备。
- 接收路径从 RX descriptor 取出已由设备填充的 mbuf，补充长度后交给 `net_rx()`。
- 描述符环大小固定为 16，索引必须按模运算循环使用。
- TX mbuf 只能在硬件设置 Descriptor Done 后回收；RX mbuf 交给协议栈后必须立即为设备换入新缓冲区。
- 发送可能来自多个进程，接收由中断触发，因此共享环和寄存器需要自旋锁保护。

官方实验说明：[MIT 6.S081 – Lab: networking](#)

1.1 实验环境

项目	配置
宿主环境	Windows + WSL2 Ubuntu, x86_64
客体系统	xv6-riscv, RISC-V 64
模拟器	QEMU 10.2.1, <code>virt</code> 机器与 E1000 设备
编译工具链	<code>riscv64-linux-gnu-gcc</code>
网络模式	QEMU user-mode network, xv6 地址 10.0.2.15
官方基线	<code>origin/net</code> , 提交 <code>1bd9c80</code>
实验成果	<code>net</code> 分支, 提交 <code>a611d08</code>

1.2 修改文件

文件	作用
<code>kernel/e1000.c</code>	实现 TX/RX 描述符环和并发控制
<code>Makefile</code>	适配 RISC-V 工具链与新版 QEMU E1000 ROM 行为
<code>gradelib.py</code>	适配 Python 3 的命令引用函数
<code>time.txt</code>	记录实验用时

2. E1000 与 DMA 描述符环

2.1 内存映射寄存器

PCI 初始化把 E1000 的寄存器区域映射到内核地址空间，`regs` 是该区域的基址。驱动按 32 位寄存器数组访问设备，其中本实验最关键的寄存器是：

寄存器	含义	驱动使用方式
<code>E1000_TDT</code>	Transmit Descriptor Tail	读取待填 TX 项；提交后推进一项
<code>E1000_RDT</code>	Receive Descriptor Tail	从下一项取包；换入新缓冲后推进
<code>E1000_ICR</code>	Interrupt Cause Read	中断入口写入以确认中断
<code>E1000_IMS</code>	Interrupt Mask Set	初始化时启用接收完成中断

初始化函数已经配置 TX/RX 环基址、长度、收发控制位和 QEMU 网卡 MAC 地址。TX 环初始每个 descriptor 均带 `E1000_TXD_STAT_DD`，表示可立即使用；RX 环的每个 descriptor 则预先绑定一个空 mbuf，供设备 DMA 写入。

2.2 descriptor 所有权

TX 环在软件与设备之间按以下顺序转移所有权：

- 1 软件读取 `TDT` 并确认该项 `DD=1`。
- 2 软件回收这个 descriptor 上一次保存的 mbuf。
- 3 软件写入新数据地址、长度和命令，清除完成状态。
- 4 软件推进 `TDT`，设备获得该项并执行 DMA 读取。
- 5 发送完成后设备写回 `DD=1`，软件以后才能再次使用。

RX 环的方向相反：设备在可用 descriptor 指向的 mbuf 中 DMA 写入包，记录长度并设置 `DD`；软件看到 `DD` 后取得旧 mbuf，给 descriptor 换入新的空 mbuf 并清零状态，再推进 `RDT` 把该项归还设备。

环索引均使用 $(index + 1) \% RING_SIZE$ ，从第 15 项正确回绕到第 0 项。官方多进程测试会收发远多于 16 个包，因此一次性线性处理无法通过。

2.3 内存可见性

描述符与数据均由 CPU 和设备共享。软件必须先完成 descriptor 字段写入，再更新 `tail` 寄存器。实现使用 `__sync_synchronize()` 放在 `tail` 更新之前，防止编译器或处理器把 MMIO 通知重排到描述符初始化之前；否则设备可能观察到尚未完整填写的项。

3. 发送路径实现

`e1000_transmit()` 的输入 `mbuf` 已包含完整 Ethernet frame。实现首先在锁内读取硬件期望的 `TDT` 位置，并检查该 `descriptor` 是否完成：

```
[c]
acquire(&e1000_lock);

index = regs[E1000_TDT];
desc = &tx_ring[index];
if((desc->status & E1000_TXD_STAT_DD) == 0){
    release(&e1000_lock);
    return -1;
}
```

若 `DD` 未设置，说明环已满或硬件还没有发送完该项。函数返回 `-1`，但不释放传入的 `mbuf`；调用者 `net_tx_eth()` 根据失败返回值执行 `mbuffree()`，从而保持清晰的所有权约定。

`descriptor` 可用时，先回收它上一次保存的 `mbuf`，再提交新包：

```
[c]
if(tx_mbufs[index])
    mbuffree(tx_mbufs[index]);

tx_mbufs[index] = m;
desc->addr = (uint64)m->head;
desc->length = m->len;
desc->cmd = E1000_TXD_CMD_EOP | E1000_TXD_CMD_RS;
desc->status = 0;

__sync_synchronize();
regs[E1000_TDT] = (index + 1) % TX_RING_SIZE;
```

`EOP` 表示当前 `descriptor` 是该帧最后一段，本实验每个包只占一个 `descriptor`；`RS` 要求设备发送后写回状态，使软件能通过 `DD` 判断完成。`mbuf` 指针保存在 `tx_mbufs[index]`，不能在提交时立即释放，因为 `DMA` 可能仍在读取其数据。

锁保护 `TDT`、`TX ring` 和 `tx_mbufs` 的复合更新。如果两个进程同时发送而没有锁，它们可能读到同一个 `TDT`，后一次写入覆盖前一次 `descriptor`，造成丢包或 `mbuf` 生命周期错误。

4. 接收路径实现

4.1 扫描接收环

接收中断调用 `e1000_recv()`。驱动从 RDT 后一项开始扫描，只要 descriptor 的 DD 仍为 1 就继续处理：

```
[c]
for(;;){
    acquire(&e1000_lock);
    index = (regs[E1000_RDT] + 1) % RX_RING_SIZE;
    desc = &rx_ring[index];
    if((desc->status & E1000_RXD_STAT_DD) == 0){
        release(&e1000_lock);
        break;
    }
    /*      descriptor      RDT */
}
```

一次中断可能对应多个已完成 descriptor，因此不能只处理一包。循环直到下一项没有 DD，可以把一次突发接收中积累的所有包交给协议栈。

4.2 换入新缓冲区

已接收 mbuf 的 len 初始为 0，设备实际接收长度记录在 descriptor 中。驱动先分配 replacement，再交换所有权：

```
[c]
replacement = mbufalloc(0);
if(replacement == 0){
    release(&e1000_lock);
    break;
}

m = rx_mbufs[index];
m->len = desc->length;
rx_mbufs[index] = replacement;
desc->addr = (uint64)replacement->head;
desc->status = 0;

__sync_synchronize();
regs[E1000_RDT] = index;
```

先成功分配 replacement，再修改 descriptor，可保证内存不足时旧包和硬件状态仍保持一致。清除状态后推进 RDT，设备以后回绕到该项时可把下一包写入新的 mbuf。

4.3 锁外调用协议栈

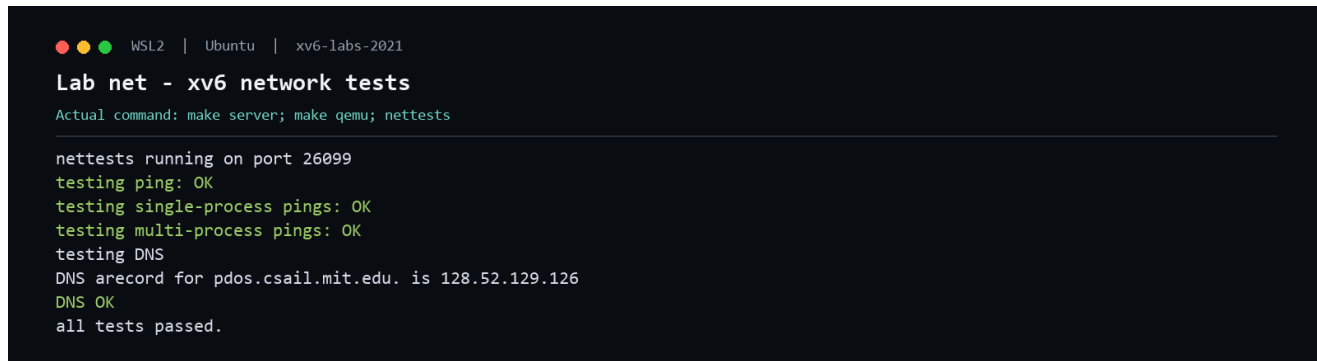
完成环状态更新后释放 `e1000_lock`，再执行 `net_rx(m)`。这是并发正确性的关键：接收到 ARP request 时，`net_rx()` 会构造 ARP reply 并调用 `e1000_transmit()`；如果仍持有同一把锁，当前 CPU 会再次申请 `e1000_lock`，形成不可恢复的自锁。

因此锁只覆盖设备共享状态，不覆盖协议处理。descriptor 已在调用 `net_rx()` 前换入新缓冲并归还设备，释放锁后其他 CPU 或下一次中断也不会重复处理同一包。

5. 实验结果与分析

5.1 nettests 运行结果

官方评分启动宿主 UDP server，并在 xv6 内运行 nettests。实际输出如下：



```

WSL2 | Ubuntu | xv6-labs-2021
Lab net - xv6 network tests
Actual command: make server; make qemu; nettests

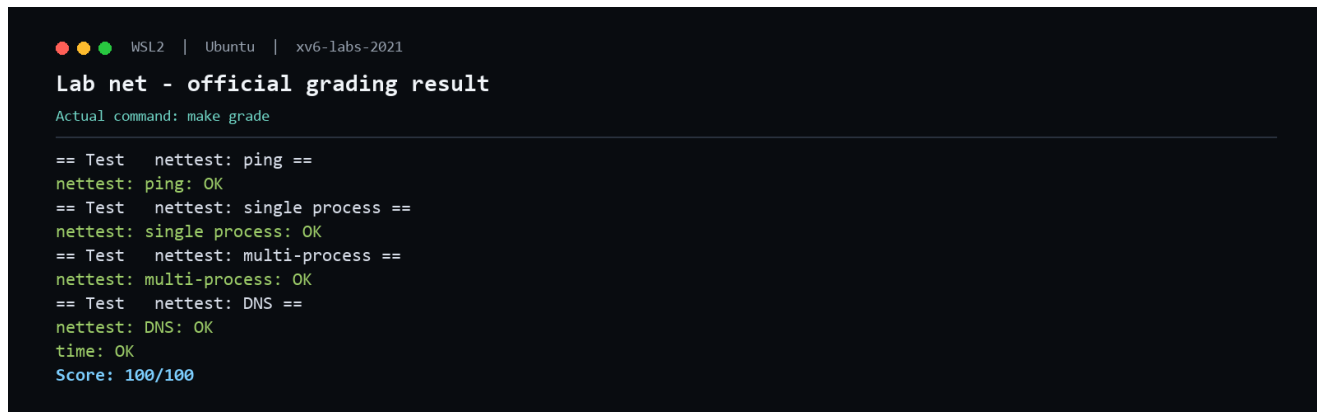
nettests running on port 26099
testing ping: OK
testing single-process pings: OK
testing multi-process pings: OK
testing DNS
DNS arecord for pdos.csail.mit.edu. is 128.52.129.126
DNS OK
all tests passed.
  
```

图 1 xv6 中 ping、并发 UDP 与 DNS 测试结果

测试覆盖单次 UDP 往返、同一进程连续收发、多个进程并发收发以及经 QEMU user-mode network 访问外部 DNS。DNS 成功解析 `pdos.csail.mit.edu`，说明发送、接收、ARP 响应、IP/UDP 处理和中断驱动流程均能持续工作。

生成的 `packets.pcap` 中可以检索到 `a message from xv6!` 与 `this is the host!`，证明 guest 到 host 和 host 到 guest 两个方向的 UDP 数据均被捕获；多次重复消息也验证 descriptor 环在超过 16 包后可以正确回绕。

5.2 官方评分



```

WSL2 | Ubuntu | xv6-labs-2021
Lab net - official grading result
Actual command: make grade

== Test  nettest: ping ==
nettest: ping: OK
== Test  nettest: single process ==
nettest: single process: OK
== Test  nettest: multi-process ==
nettest: multi-process: OK
== Test  nettest: DNS ==
nettest: DNS: OK
time: OK
Score: 100/100
  
```

图 2 官方 grade-lab-net 评分结果

各项得分为：Ping 40/40、Single process 20/20、Multi-process 20/20、DNS 19/19、Time 1/1，合计 100/100。

6. 兼容性、复现与总结

6.1 本地环境兼容处理

本机 QEMU 10.2.1 会为 E1000 尝试加载默认的 `efi-e1000.rom`，而 WSL 安装中未提供该 ROM，导致最初评分在启动 QEMU 前失败：

```
[text]
failed to find romfile "efi-e1000.rom"
```

xv6 使用 `-bios none`，实验也不通过网卡启动，因此该 option ROM 完全不参与驱动功能。Makefile 将设备参数改为：

```
[make]
-device e1000,netdev=net0,bus=pcie.0,romfile=
```

显式禁用 ROM 后，QEMU 正常创建设备并通过全部测试。另补充 `rv64gc` 架构参数、当前 GCC 的警告兼容选项，并把 Python 3 已移除的 `pipes.quote` 替换成 `shlex.quote`；这些修改不改变实验算法或评分逻辑。

6.2 官方评分复现

```
[sh]
wsl -d Ubuntu
cd /mnt/c/Users/Aemeath/Helper/2026/OS_Design/xv6-labs-2021
git switch net
make clean
make grade
echo $?
```

通过标准是最后输出 `Score: 100/100`，且退出码为 0。若要手工观察运行过程，可分别启动宿主 `server` 和 `xv6`：

```
[sh]
#
make server

#
make qemu
# xv6 shell    nettests
```

6.3 实验总结

本实验完成了 E1000 的 TX/RX DMA 描述符环驱动。实现通过 `DD` 位明确 CPU 与设备的所有权，用 `tail` 寄存器提交或归还 `descriptor`，用 `mbuf` 数组保证 DMA 期间的数据生命周期，并以自旋锁保护多进程与中断并发。接收路径在锁外调用协议栈，避免 ARP 回复重入发送路径造成死锁。最终 `ping`、单进程、多进程、DNS 和时间测试全部通过，官方得分 100/100。

xv6 Labs 2021 实验报告（八）

Lab lock: Parallelism and Locking

作者: Shalom0919

实验分支: lock

本地提交: 58ff314

代码仓库: github.com/Shalom0919/xv6_labs

完成日期: 2026-08-17

官方评分: 70/70

1. 实验概述

本实验基于 MIT 6.S081 Fall 2021 的 [xv6-labs-2021](#) 仓库 [lock](#) 分支，通过重新设计物理页分配器和磁盘块缓存的数据结构，降低多核环境下的自旋锁争用。原始实现分别使用一把 [kmem.lock](#) 保护全局空闲页链表、一把 [bcache.lock](#) 保护全部缓存 buffer；即使多个 CPU 操作互不相关的页面或磁盘块，也必须串行执行。

实验目标不是删除同步，而是缩小共享状态和锁的覆盖范围：

- 为每个 CPU 建立独立物理页 freelist，使常规 [kalloc\(\)](#) 和 [kfree\(\)](#) 只访问本地状态。
- 当本地 freelist 为空时，从其他 CPU 批量窃取页面，保持所有物理内存仍可使用。
- 使用 13 个哈希桶组织 buffer cache，每桶一把锁，使不同磁盘块的查找、引用和释放可以并行。
- 仅在 cache miss 和淘汰时使用全局锁，保持“同一磁盘块最多一个缓存副本”的不变量。
- 用释放时间戳替代原有全局 LRU 双向链表，避免 [brelse\(\)](#) 获取全局锁。

官方实验说明：[MIT 6.S081 – Lab: locks](#)

1.1 实验环境

项目	配置
宿主环境	Windows + WSL2 Ubuntu, x86_64
客体系统	xv6-riscv, RISC-V 64
QEMU CPU 数	3
编译工具链	riscv64-linux-gnu-gcc
官方基线	origin/lock , 提交 281b66c
实验成果	lock 分支, 提交 58ff314
官方评分	<code>make grade</code> , 70/70

1.2 修改文件

文件	主要作用
<code>kernel/kalloc.c</code>	per-CPU freelist 与批量 stealing
<code>kernel/bio.c</code>	哈希桶 buffer cache 与串行淘汰
<code>kernel/buf.h</code>	为 buffer 增加最后释放时间戳
<code>Makefile</code> 、 <code>gradelib.py</code>	当前工具链与 Python 兼容处理
<code>time.txt</code>	记录实验用时

2. Per-CPU 物理页分配器

2.1 数据结构拆分

原始 `kmem` 只有一把锁和一条链表。修改后按 `NCPU` 建立数组：

```
[c]
struct {
    struct spinlock lock;
    struct run *freelist;
} kmem[NCPU];

void kinit(void)
{
    for(int i = 0; i < NCPU; i++)
        initlock(&kmem[i].lock, "kmem");
    freerange(end, (void*)PHYSTOP);
}
```

所有锁名以 `kmem` 开头，使实验提供的锁统计系统调用能够识别。启动阶段由执行 `freerange()` 的 CPU 接收全部空闲页；其他 CPU 第一次分配时通过 `stealing` 获得本地页面。

2.2 本地释放与分配

`cpuid()` 只有在中断关闭时才能安全使用，否则获取 CPU 编号后进程可能被中断并迁移。实现用一层 `push_off()/pop_off()` 包围 CPU 编号和本地 `freelist` 操作：

```
[c]
push_off();
id = cpuid();
acquire(&kmem[id].lock);
r->next = kmem[id].freelist;
kmem[id].freelist = r;
release(&kmem[id].lock);
pop_off();
```

`acquire()` 自身也会关闭中断，但外层 `push_off()` 保证从 `cpuid()` 到锁释放期间 CPU 身份不变。`xv6` 的中断关闭计数支持嵌套，因此内外两层会按相反顺序安全恢复。

`kalloc()` 首先只锁当前 CPU 的 `freelist`，取出头结点后立即释放。绝大多数分配和释放由运行该进程的 CPU 本地完成，三个 CPU 不再持续竞争同一 `cache line`。

2.3 正确性不变量

每个空闲物理页恰好属于一条 `freelist`；已分配页面不在任何 `freelist` 中。链表修改只在所属 `kmem[i].lock` 下发生。页面仍保留原有地址范围、页对齐检查和 `junk fill`，因此非法释放、悬空引用检测及调用接口均与原实现一致。

3. 空闲页窃取策略

3.1 为什么采用批量窃取

如果本地为空时每次只从其他 CPU 取一页，进程连续扩展地址空间会为每一页访问 donor 锁，重新制造争用。本实现统计 donor freelist 长度，并一次摘取约一半：

```
[c]
acquire(&kmem[donor].lock);
count = 0;
for(last = kmem[donor].freelist; last; last = last->next)
    count++;

take = (count + 1) / 2;
stolen = kmem[donor].freelist;
last = stolen;
for(int n = 1; n < take; n++)
    last = last->next;
kmem[donor].freelist = last->next;
last->next = 0;
release(&kmem[donor].lock);
```

随后在本地锁下接入 stolen 链，并立即取出一页返回。批量转移使后续分配走本地快速路径，同时给 donor 保留约一半页面，避免一次窃取造成极端失衡。

3.2 避免多锁死锁

实现不同时持有 donor 与 local 两把锁：先在 donor 锁下摘链，释放后再获取 local 锁接入。这避免两个 CPU 同时互相窃取时形成 CPU0 lock -> CPU1 lock 与 CPU1 lock -> CPU0 lock 的环形等待。

摘下的链在两个锁之间暂时由当前执行流独占，不属于任何共享 freelist，因此其他 CPU 无法访问。外层中断关闭又保证当前 CPU 不会在中间阶段切换到其他内核执行路径。

3.3 内存完整性验证

kallocetest test2 会分配几乎全部物理页，并统计可用页数。本次运行报告 32495 个空闲页（总物理页 32768，差值为内核占用），测试通过。usertests sbrkmuch 也通过，说明页面在各 CPU freelist 间转移后没有丢失、重复或形成链表环。

4. 哈希化 Buffer Cache

4.1 从全局 LRU 链表到哈希桶

buffer cache 使用 13 个桶，质数桶数可降低连续 block number 的冲突：

```
[c]
#define NBUCKET 13

struct bucket {
    struct spinlock lock;
    struct buf *head;
};

struct {
    struct spinlock lock;        // only serializes cache misses
    struct buf buf[NBUF];
    struct bucket bucket[NBUCKET];
    uint clock;
} bcache;

static uint bhash(uint dev, uint blockno)
{
    return (dev + blockno) % NBUCKET;
}
```

每个 `bcache.bucket` 锁保护该桶链表、buffer 的 `dev/blockno` 身份、`refcnt` 和 `timestamp`。缓存命中只遍历并锁定目标桶，不访问全局 `bcache.lock`；映射到不同桶的磁盘块可以在不同 CPU 上并行。

4.2 命中路径

```
[c]
index = bhash(dev, blockno);
acquire(&bcache.bucket[index].lock);
for(b = bcache.bucket[index].head; b; b = b->next){
    if(b->dev == dev && b->blockno == blockno){
        b->refcnt++;
        release(&bcache.bucket[index].lock);
        acquiresleep(&b->lock);
        return b;
    }
}
release(&bcache.bucket[index].lock);
```

桶自旋锁只保护元数据操作，buffer 内容仍由原有 `sleeplock` 保护。必须在释放桶锁之前增加 `refcnt`；否则 cache miss 路径可能把刚查到但尚未锁定的 buffer 当成空闲项淘汰。

4.3 `brelse`、`bpin` 与 `bunpin`

`brelse()` 先释放 buffer

`sleeplock`，再在对应桶锁下递减引用数。引用数归零时，用原子递增的逻辑时钟记录最后释放时间：

```
[c]
b->refcnt--;
if(b->refcnt == 0)
    b->timestamp = __sync_add_and_fetch(&bcache.clock, 1);
```

原实现每次释放都要修改全局 LRU 双向链表；时间戳方案把操作限制在一个桶内。`bpin()` 和 `bunpin()` 同样改用 buffer 所在桶的锁，日志系统对 buffer 的固定引用仍保持正确。

5. Cache Miss、淘汰与并发正确性

5.1 二次查找保证唯一副本

首次桶查找 miss 后获取全局 `bcache.lock`，然后再次查找目标桶。原因是等待全局锁期间，另一个 CPU 可能已经为相同 (`dev`, `blockno`) 创建缓存项。如果不二次检查，两个 CPU 会各自选择 victim，破坏“同一块最多一个缓存副本”的核心不变量。

全局锁仅串行化 miss/eviction，不参与 hit 和 release，因此 `bcachetest test0` 的常见路径仍保持并行。

5.2 选择 LRU victim

在全局 miss 锁下扫描固定 `bcache.buf` 数组。对每个 buffer 获取它当前所属桶锁，寻找 `refcnt==0` 且 `timestamp` 最小的项。选中后重新获取旧桶锁并再次检查 `refcnt`；如果期间被命中，就放弃并重新选择。

淘汰过程从旧桶链表删除 victim，设置新 `dev`、`blockno`、`valid=0`、`refcnt=1`，再插入目标桶。旧桶和新桶相同时只获取一次锁；不同时在全局 miss 锁下获取两把桶锁。因为其他路径最多持有一把桶锁，其他 eviction 又被全局锁串行化，所以不会形成桶锁之间的循环等待。

5.3 refcnt 与 sleeplock 的配合

`refcnt` 表示使用或等待该 buffer 的调用者数；buffer `sleeplock` 确保同一时刻只有一个调用者读写内容。cache hit 在等待 `sleeplock` 前增加引用，释放内容锁后才减少引用，因此淘汰器永远不会复用正在使用或等待的 buffer。

该设计允许同一磁盘块上的调用者发生合理串行化，也允许 cache miss 淘汰阶段短暂串行化；官方性能要求关注的是不同缓存块的高频命中与释放不应争用一把全局锁。

6. 实验结果与分析

6.1 锁争用统计

```

WSL2 | Ubuntu | xv6-labs-2021
Lab lock - contention measurements
Actual command: make qemu; kallocetest; bcachetest

$ kallocetest
start test1
lock: kmem: #test-and-set 0 #acquire() 47953
lock: kmem: #test-and-set 0 #acquire() 185515
lock: kmem: #test-and-set 0 #acquire() 199554
tot= 0
test1 OK
test2 OK

$ bcachetest
start test0
lock: bcache.bucket: #test-and-set 6 #acquire() 6563
tot= 6
test0: OK
test1 OK

```

图 1 kallocetest 与 bcachetest 实际争用计数

#test-and-set 统计 acquire() 自旋循环中设置锁失败的次数。三个活跃 kmem 锁分别进行了大量获取，但失败次数均为 0，合计 tot=0。buffer cache 的 13 个桶中仅一个桶出现 6 次失败，合计 tot=6，远低于官方允许的 500。

专项测试均通过：kallocetest test1/test2 验证低争用与物理页完整性，usertests sbrkmuch 验证大地址空间分配，bcachetest test0/test1 验证低争用和超过 NBUF 后的替换路径。

6.2 官方评分

```

WSL2 | Ubuntu | xv6-labs-2021
Lab lock - official grading result
Actual command: make grade

kallocetest: test1: OK
kallocetest: test2: OK
kallocetest: sbrkmuch: OK (10.6s)
bcachetest: test0: OK
bcachetest: test1: OK
usertests: OK (176.8s)
time: OK
Score: 70/70

```

图 2 官方 grade-lab-lock 评分结果

官方脚本还执行完整 usertests，176.8 秒后输出 OK。最终各项得分为：kalloc 相关测试 30/30，bcache 相关测试 20/20，完整 usertests 19/19，time 1/1，合计 70/70。

7. 复现方法与实验总结

7.1 官方评分复现

```
[sh]
wsl -d Ubuntu
cd /mnt/c/Users/Aemeath/Helper/2026/OS_Design/xv6-labs-2021
git switch lock
make clean
make grade
echo $?
```

通过标准是最后输出 **Score: 70/70**，且退出码为

0。由于锁争用计数受宿主负载影响，应尽量关闭其他高负载程序，并保留默认 **CPUS=3**。

手工查看详细统计：

```
[sh]
make qemu
# xv6 shell
kalloctest
bcachetest
usertests sbrkmuch
```

编译兼容方面，当前 RISC-V 工具链需要明确使用 **rv64gc**，并对实验基线触发的新版本 GCC 警告进行兼容；评分库使用 **shlex.quote** 代替 Python 3 已移除的 **pipes.quote**。这些修改不改变锁算法和评分逻辑。

7.2 实验总结

本实验说明减少锁争用通常需要同时改变锁粒度和数据布局。物理页分配器通过 per-CPU ownership 把共享 freelist 转化为本地快速路径，只在资源不平衡时批量窃取；buffer cache 无法按 CPU 私有化，因此使用哈希桶分散不同磁盘块的元数据访问，并把全局锁限制在少见的 miss/eviction 路径。实现同时保持页面唯一归属、缓存块唯一副本、引用计数与 sleeplock 生命周期等不变量。最终 kmem 争用为 0、bcache 争用为 6，所有专项和回归测试通过，官方得分 70/70。

xv6 Labs 2021 实验报告（九）

Lab fs: File System

作者: Shalom0919

实验分支: fs

本地提交: ebd5b99

代码仓库: github.com/Shalom0919/xv6_labs

完成日期: 2026-08-17

官方评分: 100/100

1. 实验概述

本实验基于 MIT 6.S081 Fall 2021 的 `xv6-labs-2021` 仓库 `fs` 分支，围绕 `xv6` 文件系统完成两个功能：扩大单个文件可寻址的数据块数量，以及实现符号链接。实验涉及磁盘 `inode` 格式、逻辑块到磁盘块的映射、文件截断回收、路径解析和系统调用接口。

主要目标如下：

- 将 `inode` 的 12 个直接块指针调整为 11 个，以直接、一级间接和二级间接寻址把最大文件从 268 块提升到 65,803 块。
- 在 `itrunc()` 中完整回收二级间接树，避免磁盘块泄漏。
- 新增 `symlink(target, path)`、`T_SYMLINK` 和 `O_NOFOLLOW`，在 `open()` 中递归跟随并检测链接环。
- 保持 `link()` 与 `unlink()` 操作链接 `inode` 本身，不改变原有硬链接语义。

官方实验说明：[MIT 6.S081 - Lab: file system](#)

1.1 实验环境

项目	配置
宿主环境	Windows + WSL2 Ubuntu, x86_64
客体系统	<code>xv6-riscv</code> , RISC-V 64
文件系统镜像	200,000 blocks, <code>BSIZE=1024</code>
编译工具链	<code>riscv64-linux-gnu-gcc</code>
官方基线	<code>origin/fs</code> , 提交 <code>46bcba9</code>
实验成果	<code>fs</code> 分支, 提交 <code>ebd5b99</code>
官方评分	<code>make grade</code> , 100/100

1.2 修改文件

文件	主要作用
<code>kernel/fs.h</code> , <code>kernel/file.h</code>	调整 <code>inode</code> 地址槽并定义最大文件大小
<code>kernel/fs.c</code>	实现二级间接映射和完整块回收
<code>kernel/sysfile.c</code>	实现 <code>sys_symlink()</code> 和 <code>open()</code> 链接跟随
系统调用与用户接口头文件	注册调用，新增链接类型、标志和用户桩函数
<code>Makefile</code> , <code>gradelib.py</code>	构建 <code>symlinktest</code> 并兼容当前工具链
<code>time.txt</code>	记录实验用时

2. 大文件寻址结构

2.1 inode 地址槽重新分配

磁盘 inode 的大小不能改变。原布局为 12 个直接块地址和 1 个一级间接块地址，共 13 个 `uint` 槽位。本实现将其重新解释为 11 个直接块、1 个一级间接块和 1 个二级间接块：

```
[c]
#define NDIRECT 11
#define NINDIRECT (BSIZE / sizeof(uint))
#define NDOUBLY (NINDIRECT * NINDIRECT)
#define MAXFILE (NDIRECT + NINDIRECT + NDOUBLY)

struct dinode {
    ...
    uint addrs[NDIRECT+2];
};
```

内存 inode 的 `addrs[]` 同样改为 `NDIRECT+2`，确保内存表示和磁盘表示具有相同大小。每个间接块可以保存 $1024 / 4 = 256$ 个块号，因此最大文件块数为：

层级	数据块数量	逻辑块范围
直接块	11	0 至 10
一级间接	256	11 至 266
二级间接	256 x 256	267 至 65,802
合计	65,803	最大约 64.26 MiB

2.2 二级索引计算

进入二级间接区后，先减去直接区和一级间接区的逻辑块数，再将剩余编号拆成两级索引：

```
[c]
bn -= NINDIRECT;
uint outer = bn / NINDIRECT;
uint inner = bn % NINDIRECT;
```

`outer` 选择二级间接块中的一级间接块地址，`inner` 再选择该一级间接块中的数据块地址。除法与取模将连续逻辑块稳定映射到 256×256 的地址矩阵。

3. bmap() 二级间接映射

3.1 按需分配元数据块

`bmap()` 只有在实际写入对应范围时才分配二级间接块、下层一级间接块和数据块：

```
[c]
if((addr = ip->addrs[NDIRECT+1]) == 0)
    ip->addrs[NDIRECT+1] = addr = balloc(ip->dev);
bp = bread(ip->dev, addr);
a = (uint*)bp->data;

if((addr = a[outer]) == 0){
    a[outer] = addr = balloc(ip->dev);
    log_write(bp);
}
brelse(bp);

ibp = bread(ip->dev, addr);
ia = (uint*)ibp->data;
if((addr = ia[inner]) == 0){
    ia[inner] = addr = balloc(ip->dev);
    log_write(ibp);
}
brelse(ibp);
return addr;
```

该流程保持稀疏使用时的空间效率：小文件仍然只使用直接块；文件越过第 267 个数据块后才创建二级结构。`balloc()` 返回清零后的块，因此新建索引块中未使用的槽位均为 0。

3.2 日志与 buffer 生命周期

每次修改索引块内的块号后调用 `log_write()`，使元数据更新进入 xv6 文件系统日志。所有 `bread()` 都存在配对的 `brelse()`；在读取下一级前先记录块号并释放上一级 buffer，避免不必要地长期占用 buffer cache。

直接写 `ip->addrs[NDIRECT+1]` 不需要单独调用 `log_write()`，因为 inode 的内存副本会在 `writei()` 完成后由原有 `iupdate()` 路径写回并记录到日志中。这与原一级间接块入口的处理方式一致。

4. itrunc() 完整回收

删除或截断大文件时，除了数据块，还必须释放两层索引块。实现先读取顶层二级间接块，再逐项读取其中存在的一级间接块：

```
[c]
if(ip->addrs[NDIRECT+1]){
    bp = bread(ip->dev, ip->addrs[NDIRECT+1]);
    a = (uint*)bp->data;
    for(i = 0; i < NINDIRECT; i++){
        if(a[i]){
            ibp = bread(ip->dev, a[i]);
            ia = (uint*)ibp->data;
            for(j = 0; j < NINDIRECT; j++){
                if(ia[j])
                    bfree(ip->dev, ia[j]);
            }
            brelse(ibp);
            bfree(ip->dev, a[i]);
        }
    }
    brelse(bp);
    bfree(ip->dev, ip->addrs[NDIRECT+1]);
    ip->addrs[NDIRECT+1] = 0;
}
```

释放顺序由叶到根：先释放数据块，再释放一级索引块，最后释放顶层二级索引块。若先释放父索引块，就会丢失对子块的地址引用。完成后将 inode 地址槽清零，并由原有代码把 `ip->size` 设为 0、调用 `iupdate()` 持久化。

`bigfile` 在写完 65,803 块后会删除测试文件，随后完整 `usertests` 继续创建和写入文件。两者连续通过，说明大文件使用的块已被正确归还，没有产生可见的空间泄漏或重复分配。

5. 符号链接系统调用

5.1 用户态到内核态接口

实现新增系统调用号 `SYS_symlink=22`，并在 `user/usys.pl`、`user/user.h`、`kernel/syscall.c` 中接通用户态桩函数和内核分发表。`T_SYMLINK=4` 标识链接 `inode`，`O_NOFOLLOW=0x800` 与现有 `open` 标志不重叠。

5.2 创建链接

`sys_symlink()` 不要求 `target` 已存在。它创建类型为 `T_SYMLINK` 的 `inode`，并将目标路径连同字符串终止符保存在 `inode` 数据中：

```
[c]
if(argstr(0, target, MAXPATH) < 0 ||
    argstr(1, path, MAXPATH) < 0)
    return -1;

begin_op();
if((ip = create(path, T_SYMLINK, 0, 0)) == 0){
    end_op();
    return -1;
}
n = strlen(target) + 1;
if(writei(ip, 0, (uint64)target, 0, n) != n){
    iunlockput(ip);
    end_op();
    return -1;
}
iunlockput(ip);
end_op();
```

创建 `inode`、写入目标和目录项更新都位于一个文件系统事务中。保存终止符使后续 `namei()` 可以直接使用读取出的缓冲区。悬空链接能够成功创建，只有在普通 `open()` 尝试解析不存在的 `target` 时才返回失败。

6. open() 路径跟随与并发语义

6.1 递归跟随和环检测

当未指定 `O_NOFOLLOW` 且当前 `inode` 类型为 `T_SYMLINK` 时, `open()` 读取目标路径, 释放当前 `inode`, 再对目标调用 `namei()`:

```
[c]
if((omode & O_NOFOLLOW) == 0){
    for(int depth = 0; ip->type == T_SYMLINK; depth++){
        if(depth >= 10 ||
            (n = readi(ip, 0, (uint64)target, 0, MAXPATH)) <= 0){
            iunlockput(ip);
            end_op();
            return -1;
        }
        target[MAXPATH-1] = 0;
        iunlockput(ip);
        if((ip = namei(target)) == 0){
            end_op();
            return -1;
        }
        ilock(ip);
    }
}
```

最多跟随 10 层链接。若第 10 层后仍得到符号链接, 则按循环或异常深链处理并返回

-1。每次解析下一目标前使用 `iunlockput()`, 避免在 `namei()` 遍历文件系统时持有旧 `inode` 锁, 从而防止自环和多节点环造成锁递归或死锁。

6.2 O_NOFOLLOW 与其他系统调用

指定 `O_NOFOLLOW` 时跳过解析, 文件描述符直接引用符号链接 `inode`, 因此 `fstat()` 能观察到 `T_SYMLINK`。`link()` 和 `unlink()` 没有调用这段跟随逻辑, 仍操作路径最后一个分量对应的 `inode` 本身, 满足实验要求。

并发测试由两个子进程反复创建、查询和删除同一路径。`sys_symlink()` 和 `sys_unlink()` 均使用 `begin_op()/end_op()`, 目录项修改仍在原有 `inode` 锁和日志规则下执行; 测试通过说明没有暴露半写入的链接 `inode`, 也没有破坏目录一致性。

7. 实验结果、复现与总结

7.1 官方评分结果

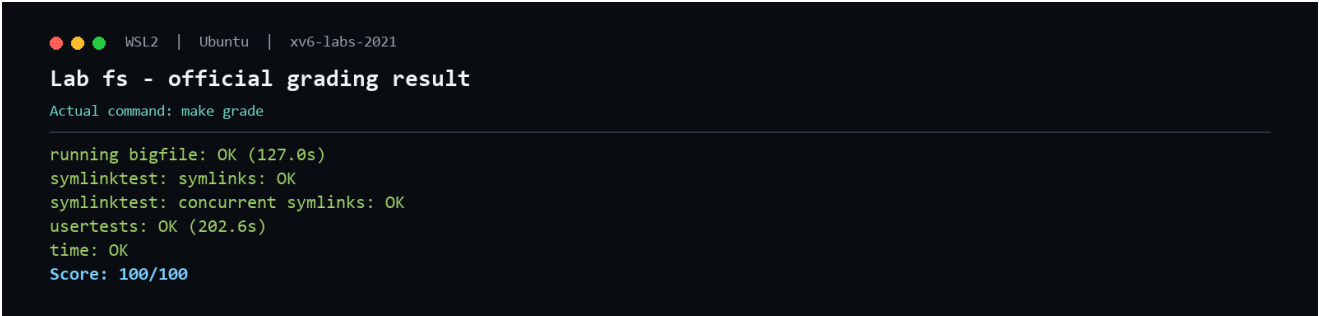


图 1 官方 `grade-lab-fs` 评分结果

官方脚本在 WSL 的 Linux 原生文件系统中运行，结果如下：

测试项	得分	结果	分析
bigfile	40/40	127.0 秒, OK	成功写入 65,803 块并完成清理
基本符号链接	20/20	OK	跟随、悬空链接、深链、环和 <code>O_NOFOLLOW</code> 正确
并发符号链接	20/20	OK	并发创建、查询和删除保持一致性
usertests	19/19	202.6 秒, OK	原有文件系统及系统调用无回归
time.txt	1/1	OK	格式符合要求
合计	100/100	通过	官方评分脚本退出码为 0

首次直接在 `/mnt/c` 的 Windows 挂载卷运行时，`bigfile` 和 `usertests` 分别触发 180 秒、360 秒超时；输出始终停留在正常写入进度，没有 panic 或断言错误。将同一工作树复制到 WSL 的 `ext4` 文件系统后，不修改代码和评分脚本即得到 100/100。原因是本实验会密集随机更新约 200 MB 的 `fs.img`，跨 Windows 挂载层的同步 I/O 显著慢于 Linux 原生文件系统。

7.2 官方评分复现

建议在 WSL 原生临时目录运行官方评分，避免 `/mnt/c` 性能造成假失败：

```
[sh]
wsl -d Ubuntu
workdir=$(mktemp -d)
cp -a /mnt/c/Users/Aemeath/Helper/2026/OS_Design/xv6-labs-2021/. "$workdir/"
cd "$workdir"
git switch fs
make clean
make grade
echo $?
```

通过标准是最后输出 `Score: 100/100`，并且 `echo $?` 输出 `0`。此命令仍执行仓库原始的 `grade-lab-fs`，只改变工作目录所在的存储介质，不修改超时、测试内容或计分规则。

7.3 实验总结

本实验将文件系统的两条关键路径连接起来：`bmap()` 负责从逻辑块号向下建立地址树，`itrunc()` 必须按相反方向完整拆除同一棵树；两者只有对称实现，才能同时保证大文件可用和磁盘空间可回收。符号链接则展示了 `inode` 数据与路径解析的组合：链接本身是独立 `inode`，目标路径是其内容，只有 `open()` 在未设置 `O_NOFOLLOW` 时解释该内容。最终大文件、符号链接、并发测试和完整回归测试全部通过，官方得分 100/100。

xv6 Labs 2021 实验报告（十）

Lab mmap: Memory-mapped Files

作者: Shalom0919

实验分支: mmap

本地提交: a69ed4e

代码仓库: github.com/Shalom0919/xv6_labs

完成日期: 2026-08-17

官方评分: 140/140

1. 实验概述

本实验基于 MIT 6.S081 Fall 2021 的 [xv6-labs-2021](#) 仓库 `mmap` 分支，为 `xv6` 增加文件支持的 `mmap()` 和 `munmap()`。实现允许进程把文件内容映射到虚拟地址空间，通过缺页异常按需载入页面，并根据 `MAP_SHARED` 或 `MAP_PRIVATE` 决定解除映射时是否写回文件。

主要目标如下：

- 每个进程维护 VMA 表；`mmap()` 只登记地址范围和文件引用，不提前分配物理页。
- 首次访问映射页时，由缺页处理分配页面并从正确的文件偏移读入数据。
- `munmap()` 支持整体、开头或结尾解除映射，并对共享可写页执行日志化写回。
- 在 `fork()`、`exit()` 和 `exec()` 中正确复制或释放 VMA 与文件引用。

官方实验说明：[MIT 6.S081 - Lab: mmap](#)

1.1 实验环境

项目	配置
宿主环境	Windows + WSL2 Ubuntu, x86_64
客体系统	<code>xv6-riscv</code> , RISC-V 64
页大小	4096 bytes
编译工具链	<code>riscv64-linux-gnu-gcc</code>
官方基线	<code>origin/mmap</code> , 提交 <code>2255a4c</code>
实验成果	<code>mmap</code> 分支, 提交 <code>a69ed4e</code>
官方评分	<code>make grade</code> , 140/140

1.2 修改文件

文件	主要作用
<code>kernel/proc.h</code>	定义 VMA 并加入进程结构
<code>kernel/proc.c</code>	映射、缺页载入、写回、解除映射与生命周期管理
<code>kernel/trap.c</code>	将用户页异常交给 VMA 缺页处理
<code>kernel/sysfile.c</code> 、 <code>kernel/exec.c</code>	实现调用入口并在程序替换前释放旧映射
系统调用与用户接口文件	注册调用号、分发表和用户桩函数
<code>Makefile</code> 、 <code>gradelib.py</code>	构建 <code>mmaptest</code> 并兼容当前工具链

2. VMA 数据结构与地址布局

2.1 每进程 VMA 表

每个进程最多维护 16 个映射区域。VMA 只保存描述信息，物理页是否存在由页表决定：

```
[c]
#define NVMA 16

struct vma {
    int valid;
    uint64 addr;
    uint64 length;
    int prot;
    int flags;
    struct file *file;
    uint64 offset;
};

struct proc {
    ...
    struct vma vmas[NVMA];
};
```

`file` 保存经过 `filedup()` 增加引用计数后的文件对象。因此用户在 `mmap()` 后关闭原文件描述符，映射仍然可以继续缺页读取和写回；文件被 `unlink()` 后，`inode` 也会因 VMA 引用继续存活。

2.2 高地址向下分配

普通程序映像、栈和 `heap` 从低地址向上增长。映射区从 `TRAPFRAME` 下方按页向低地址分配：

地址方向	区域
高地址	TRAMPOLINE、TRAPFRAME
向下增长	mmap VMA 区域
未使用间隔	防止 mmap 与 heap 相撞
向上增长	heap、用户栈、程序段

每次映射取所有有效 VMA 的最低起始地址作为上界，再减去页对齐后的映射长度。`growproc()` 同时检查新的 `heap` 末端不得越过最低 VMA。这一布局不把懒映射空洞计入 `p->sz`，因此原有 `uvmcopy()` 和普通地址空间释放逻辑不需要遍历未映射页。

3. mmap() 与懒分配

3.1 参数与权限检查

系统调用入口读取六个参数，并限制为实验要求的子集：`addr` 和 `offset` 必须为 0，长度必须为正，`flags` 必须恰好为 `MAP_SHARED` 或 `MAP_PRIVATE`。任何映射都需要读取原始文件；共享可写映射还要求文件以可写方式打开。

```
[c]
if(addr != 0 || length <= 0 || offset != 0)
    return -1;

return vmamap(myproc(), length, prot, flags, f, offset);
```

`MAP_PRIVATE` | `PROT_WRITE` 可以映射只读打开的文件，因为页面修改不会回写；`MAP_SHARED` | `PROT_WRITE` 若文件不可写则立即失败。这由官方 `read-only` 与 `private` 测试覆盖。

3.2 只登记，不读取

`vmamap()` 找到空闲 VMA 和无冲突地址后，仅填写元数据并执行 `filedup()`：

```
[c]
v->valid = 1;
v->addr = addr;
v->length = PGROUNDUP(length);
v->prot = prot;
v->flags = flags;
v->file = filedup(f);
v->offset = offset;
return addr;
```

此时用户页表中没有对应 PTE，也没有分配物理内存。大文件映射的建立成本与文件大小无关，只有实际访问的页面才占用内存。这同时允许映射长度大于当前物理内存。

4. 缺页处理与文件载入

4.1 trap 路由

RISC-V 的 instruction、load 和 store page fault 分别使用 `scause` 12、13、15。 `usertrap()` 将这三类异常交给 `vmafault()`；只有异常地址位于有效 VMA 且访问类型符合权限时才恢复执行，否则按非法用户访问终止进程。

```
[c]
} else if((r_scause() == 12 || r_scause() == 13 ||
          r_scause() == 15) &&
          vmafault(p, r_stval(), r_scause()) == 0){
    // A file-backed VMA page was populated lazily.
}
```

权限检查发生在分配之前。例如对只读映射执行 store 会返回失败，而不是把页面重新映射为可写。若对应 PTE 已有效但仍产生权限异常，也直接失败，避免 `mappages()` 的 remap panic。

4.2 分配、读取和映射

缺页地址先向下按页对齐，然后分配并清零一页。文件偏移由 VMA 起始偏移加上映射内页偏移得到：

```
[c]
va = PGROUNDDOWN(faultva);
mem = kalloc();
memset(mem, 0, PGSIZE);

fileoff = v->offset + (va - v->addr);
ilock(v->file->ip);
n = readi(v->file->ip, 0, (uint64)mem, fileoff, PGSIZE);
iunlock(v->file->ip);
```

文件末尾不足一页时，`readi()` 只读取存在的字节，其余部分保留为 0。PTE 权限由 `prot` 转换为 `PTE_R`、`PTE_W`、`PTE_X` 和 `PTE_U`。RISC-V 不允许 write-only 叶 PTE，因此可写页同时设置 `PTE_R`。

若读取或页表映射失败，代码释放刚分配的物理页并让进程收到访问失败，不留下孤立页面。

5. munmap() 与共享写回

5.1 部分解除映射

`vmaunmap()` 验证目标范围属于同一 VMA，并且从区域开头、结尾或整体解除映射。它逐页检查页表，只处理已经因缺页访问而存在的 PTE；从未访问的懒页面无需调用 `uvmunmap()`，从而避免原函数对不存在映射的 panic。

整体解除映射时执行 `fileclose()` 并清空 VMA。若只移除开头，则同时增加 `addr` 和文件 `offset`；若只移除结尾，则缩短 `length`：

```
[c]
if(addr == v->addr && span == v->length){
    fileclose(v->file);
    memset(v, 0, sizeof(*v));
} else if(addr == v->addr){
    v->addr += span;
    v->offset += span;
    v->length -= span;
} else {
    v->length -= span;
}
```

更新 `offset`

是开头解除映射的关键：剩余区域的新起始页仍必须对应原文件中向后移动相同长度的位置。

5.2 MAP_SHARED 写回

对于已经映射的 `MAP_SHARED | PROT_WRITE` 页面，解除映射前把物理页内容写回 VMA 对应的文件偏移。`MAP_PRIVATE` 页面直接释放，不修改文件。

单页大小为 4096 bytes，可能超过一次 xv6 日志事务允许的安全写入量。实现复用 `filewrite()` 的上限公式，把页面拆分为最多 3072 bytes 的事务：每段分别执行 `begin_op()`、inode 锁、`writel(user_src=0)` 和 `end_op()`。这样既不修改共享 `struct file` 的 `off`，又不会耗尽日志空间。

6. fork、exit 与 exec 生命周期

6.1 fork 继承

`fork()` 在复制普通低地址内存后复制所有有效 VMA，并对每个文件执行 `filedup()`:

```
[c]
for(i = 0; i < NVMA; i++){
    if(p->vmass[i].valid){
        np->vmass[i] = p->vmass[i];
        np->vmass[i].file = filedup(p->vmass[i].file);
    }
}
```

映射页位于高地址，不在 `p->sz` 范围内，因此不会被 `uvmcopy()` 立即复制。子进程第一次访问时独立分配物理页并从同一文件读取。实验明确允许父子进程不共享同一个物理页，这种设计保持实现简单，同时满足 `fork_test`。

6.2 退出和程序替换

`exit()` 在关闭普通文件描述符前调用 `vmaunmapall()`，依次写回共享映射、释放物理页并减少 VMA 文件引用。父进程之后执行 `wait()` 并最终释放页表时，不会遇到遗留的高地址叶 PTE。

`exec()` 成功构造新程序页表后、替换旧页表前同样调用 `vmaunmapall()`。这补充了官方基础测试之外的生命周期路径，避免带着映射执行 `exec()` 时泄漏文件引用或物理页。若新程序构造失败，旧映射保持不变。

7. 实验结果、复现与总结

7.1 官方评分结果

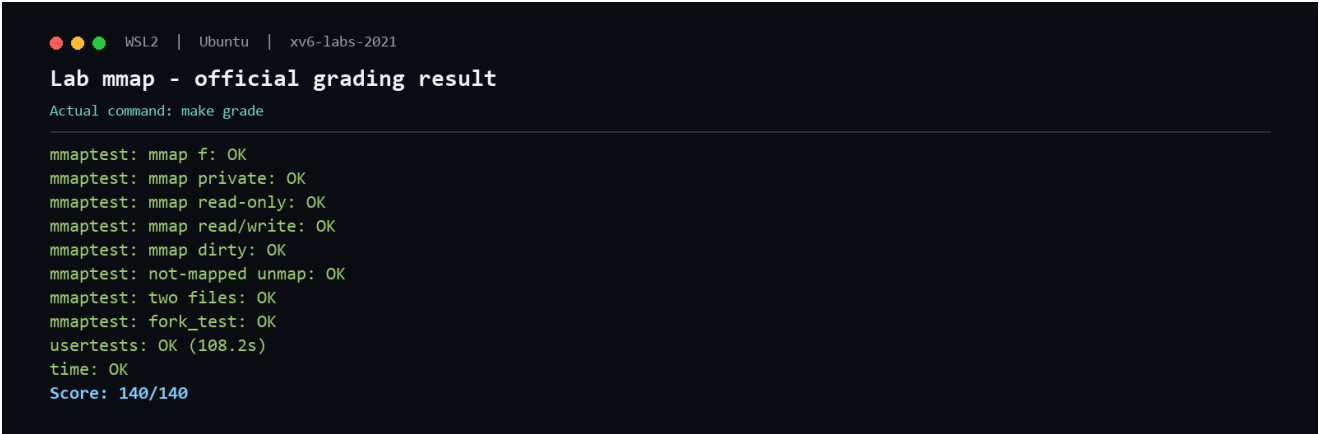


图 1 官方 grade-lab-mmap 评分结果

测试组	得分	结果与覆盖内容
基础文件映射	20/20	懒加载文件内容正确
private、read-only、read/write	30/30	私有修改和访问权限正确
dirty、partial unmap、two files	30/30	共享写回、部分解除和多 VMA 正确
fork_test	40/40	子进程继承映射和文件引用正确
usertests	19/19	108.2 秒，完整回归通过
time.txt	1/1	格式符合要求
合计	140/140	官方评分脚本退出码为 0

所有 `mmaptest` 子项在 6.3 秒内完成。完整 `usertests` 通过，说明新加入的高地址映射、缺页异常分支、退出清理和 heap 边界检查没有破坏原有进程、内存与文件系统行为。

7.2 官方评分复现

```
[sh]
wsl -d Ubuntu
cd /mnt/c/Users/Aemeath/Helper/2026/OS_Design/xv6-labs-2021
git switch mmap
make clean
make grade
echo $?
```

通过标准是最后输出 `Score: 140/140`，并且 `echo $?` 输出 `0`。手工运行时可以执行 `make qemu`，进入 `xv6 shell` 后运行 `mmaptest`；最终应输出 `mmaptest: all tests succeeded`。

7.3 实验总结

本实验把虚拟内存、异常处理、文件系统和进程生命周期连接成一条完整路径。VMA 是逻辑承诺，页表 PTE 是实际驻留状态；`mmap()` 建立承诺，缺页异常按需兑现，`munmap()` 负责写回和释放。正确性不仅取决于单次映射，还取决于文件关闭、部分解除、`fork()`、`exit()` 和 `exec()` 后引用计数与页表状态始终匹配。最终所有专项测试和回归测试通过，官方得分 140/140。